

Contents

1	Java 8 Streams Basic Coding questions	7
1.1	Filter Even And odd Numbers in a List	7
1.2	Convert List Of String To Upper Case	7
1.3	Sum Of All Numbers In A List	8
1.4	Find Maximum Number In A List	8
1.5	Sort List Of Integers	8
1.6	Remove Duplicates In A List	9
1.7	Count No Of Elements In List	9
1.8	Matching Functions	9
1.9	Count Of Each Character In String	9
1.10	Count Of Each Word In String	10
1.11	Count Strings Longer Than Five	10
1.12	Sum Numbers Greater Than Ten	11
1.13	Find First Divisible By Seven	11
1.14	List To Map Word Length	11
1.15	GroupBy FirstLetter	11
1.16	Partition Even Odd	12
1.17	Flatten List Of Lists	12
1.18	Print First Element	12
1.19	Optional Find String Starting With A	13
1.20	Sort Strings By Length Descending and Ascending	13
1.21	Collect To Set	14
1.22	Distinct Numbers	14
1.23	Filter And Sort	14
1.24	Square Each Number	14
1.25	Remove Nulls	14
1.26	Sum Of Evens	15
1.27	Reduce Concatenate Strings And Multiply Integers	15
1.28	Join With Comma	15
1.29	Top3 Highest Value	16
1.30	Skip First2	16
1.31	Debugging using Peek	16
1.32	Sum Of Squares Greater Than 10	16
1.33	Convert To Array	17
1.34	Count Frequencies	17
1.35	Unique Sorted Words	17
1.36	Longest String	17
1.37	Shortest String	17
1.38	Even Numbers Using IntStream	18
1.39	Generate First NSquares	18
1.40	Prime Check Using Streams	18

1.41	Find Palindromes	18
1.42	Distinct Count Of Numbers	19
1.43	Convert List Of Integers To String	19
1.44	Filter By Prefix	19
1.45	Remove Negative Numbers	19
1.46	Double The Numbers	19
1.47	Strings To Char Stream	20
1.48	Average Of Numbers	20
1.49	Collect To Tree-Set	20
1.50	Statistics Summary	20
1.51	Find-First vs Find-Any in Parallel stream	21
1.52	Filter out empty Optionals	21
1.53	Flatten nested List<Optional<List<String>>>	21
1.54	or-Else vs orElse-Get performance pitfall	22
1.55	Short-circuit Optional with Stream Any-Match	22
2	Java 8 Tricky Interview Questions	23
2.1	Model class used for coming questions	23
2.2	Group by dept, find oldest in each group	23
2.3	Partition by age<40 and collect names	23
2.4	Lazy evaluation and short-circuiting	24
2.5	Parallel stream with reduce (sum of ages)	24
2.6	Nested collectors - average age per dept for people older than 30	24
2.7	Find duplicate ages	25
2.8	Collecting into a map with merge function (name length)	25
2.9	Find Duplicate Elements in a List	25
2.10	Find First Non-Repeating Character in a String	25
2.11	Find Common Elements Between Two Lists	26
2.12	Convert List to Map	26
2.13	Find Average Salary of Employees	26
2.14	Count Characters in String	27
2.15	Sort Employees by Salary	27
2.16	Find Employee with Second Highest Salary	27
2.17	Generate Fibonacci Using Streams	27
2.18	Collecting into a map with merge function (name length)	27
3	Map And FlatMap	29
3.1	Grouping-By with Optional values	29
3.2	Optional with max/min	29
3.3	Optional chaining with possible null objects	29
3.4	Optional inside Stream	30
3.5	Flatten Nested Optionals	30
3.6	Combined nested collections	30
3.7	FlatMap multiple streams	31
3.8	Empty Optional map vs flatMap	31
3.9	FlatMap Optional containing list	31
3.10	FlatMap to Stream (flattened)	32
3.11	Map to Stream (nested)	32
3.12	Optional map vs flatMap	32
3.13	FlatMap strings to chars (flattened)	32
3.14	Map Strings to split arrays → nested	33
3.15	Map Strings to split arrays → nested	33

3.16	flatMap flattens nested lists	33
3.17	Simple map	34
3.18	Optional stream flatten	34
3.19	Optional filter + flatMap	34
3.20	Optional map chaining	35
3.21	Optional of Optional empty	35
3.22	Optional map vs flatMap with list	35
3.23	Optional empty handling	36
3.24	Nested Optionals flatten	36
3.25	Flatten Optional of List	36
3.26	Combined nested collections + Optional flatten	37
3.27	flatMap multiple streams	37
3.28	Empty Optional map vs flatMap	37
3.29	Flatten Optional containing list	38
3.30	Stream of streams flattened	38
3.31	Stream of streams with map	38
3.32	Simple map (square numbers)	39
3.33	Nested lists with map (nested)	39
3.34	Nested lists with flatMap (flattened)	40
3.35	Map strings to split arrays (nested)	40
3.36	flatMap strings to chars (flattened)	40
3.37	Optional map vs flatMap	41
3.38	Concurrent collectors	41
3.39	Concurrent collectors	41
3.40	Parallel Map-Reduce large dataset	42
3.41	Short-circuiting parallel	42
3.42	Custom ForkJoinPool	42
3.43	Optional in parallel stream	43
3.44	Parallel ordering	43
3.45	Shared mutable state in parallel stream	43
3.46	Parallel vs sequential benchmark	44
3.47	Find top N elements efficiently	44
3.48	Sequential vs Parallel Streams	44
3.49	Lazy evaluation example	45
3.50	Primitive Streams avoid boxing/unboxing	45
3.51	Short-circuit filter with limit	45
3.52	Short-circuit filter with limit	46
3.53	Parallel stream with shared mutable state (pitfall)	46
3.54	Sorting large streams	47
3.55	Collectors efficiency	47
3.56	Limit early	47
3.57	Map/reduce optimization	48
3.58	Boxing/unboxing overhead	48
4	Employee Based Programs	49
4.1	Model class used for coming questions	49
4.2	Group by dept → gender → top 2 salaries	49
4.3	Median salary per dept	50
4.4	Flatten nested list of Optional[Employee]	50
4.5	Second highest salary	51
4.6	Top 3 salaries per dept	51

4.7	Flatten <code>List<List<String>></code> and remove duplicates	52
4.8	Count word frequency	52
4.9	Find longest string starting with vowel	52
4.10	Flatten nested map to entries	53
4.11	Top N frequent elements	53
4.12	Employees above 75th percentile	53
4.13	Optional chaining: nested field length	54
4.14	First non-empty Optional	54
4.15	Combine multiple Optionals	55
4.16	Parallel sum of squares	55
4.17	Parallel word count ignoring stop words	55
4.18	Update <code>ConcurrentHashMap</code> with merge	55
4.19	Top 10 frequent items from multiple lists	56
4.20	Flatten tree-like structure	56
4.21	Compute cumulative sum	56
4.22	Flatten nested <code>List<Optional<List<String>>></code> to distinct sorted strings	57
4.23	Fibonacci using <code>Stream.iterate</code>	57
4.24	Pagination with skip/limit	57
4.25	Prime numbers using streams	58
4.26	Short-circuit operation (<code>anyMatch</code>)	58
5	Advanced Stream Multithreading Exercises	59
5.1	Parallel ordering	59
5.2	Optional in parallel stream	59
5.3	Custom <code>ForkJoinPool</code>	59
5.4	Parallel Map-Reduce large dataset	60
5.5	Top-N elements using parallel stream	60
5.6	Flatten nested Optional lists in parallel	61
5.7	Parallel <code>findFirst</code> vs <code>findAny</code>	61
5.8	Primitive vs boxed streams performance	61
5.9	Parallel filtering timing	62
5.10	Parallel distinct performance	62
5.11	Parallel sort timing	62
5.12	Combined collectors with parallel stream	63
5.13	Nested Optional + parallel stream	63
5.14	Parallel Top-N per group	64
5.15	Custom <code>ForkJoinPool</code> with limited threads	64
5.16	Parallel <code>anyMatch</code> short-circuit	65
5.17	Parallel <code>allMatch</code> timing	65
5.18	Parallel partitioning collectors	65
5.19	Parallel <code>flatMap</code> large nested lists	66
5.20	Parallel reduce with combiner	66
5.21	Parallel stream peek for debugging	66
5.22	Parallel stream with exception handling	67
6	Real time Streams Demo	69
6.1	Model class used for coming questions	69
6.2	Active users sorted by last login	70
6.3	Total transaction amount per user	70
6.4	Out of stock products	70
6.5	Highest salary by department	71
6.6	Customers with >1 orders	71

6.7	Comma-separated emails	71
6.8	Average student scores	71
6.9	Distinct cities	72
6.10	Flatten all purchased items	72
6.11	Top 3 best selling products	72
6.12	Initialize Dataset	72
6.13	Distinct cities	75
6.14	Banking Examples	76
6.15	E-commerce Examples	77

Chapter 1

Java 8 Streams Basic Coding questions

1.1 Filter Even And odd Numbers in a List

```
static void filterEvenNumbersFromList(){
    List<Integer> nums = Arrays.asList
        (1,2,3,4,5,6,7,8,9,10);
    List<Integer> result = nums.stream()
        .filter(n -> n % 2 == 0)
        .toList();
    System.out.println("Even numbers in List: " +
        result);

    user .filter(n -> n % 2 == 0) to get odd numbers
}
```

Output

```
Even numbers in List: [2, 4, 6, 8, 10]
Odd numbers in List: [1, 3, 5, 7, 9]
```

1.2 Convert List Of String To Upper Case

```
static void convertListOfStringsToUpperCase(){
    List<String> names = Arrays.asList("john","alice","bob"
    );
    List<String> result = names.stream()
        .map(String::toUpperCase)
        .collect(Collectors.toList());
    System.out.println("To Uppercase: "+result);

    user .map(String::lowerCase) to get lower case
}
```

Output

```
To Uppercase[JOHN, ALICE, BOB]
To LowerCase [john, alice, bob]
```

1.3 Sum Of All Numbers In A List

```
static void sumOfAllNumbersInAList(){
    List<Integer> nums = Arrays.asList(1,2,3,4,5);
    int sum = nums.stream()
        .mapToInt(Integer::intValue)
        .sum();
    System.out.println("Sum of All Numbers in a list: " +
        sum);
}
```

Output

```
Sum of All Numbers in a list: 15
```

1.4 Find Maximum Number In A List

```
static void sumOfAllNumbersInAList(){
    List<Integer> nums = Arrays.asList(1,2,3,4,5);
    int sum = nums.stream()
        .mapToInt(Integer::intValue)
        .sum();
    System.out.println("Sum of All Numbers in a list: " +
        sum);
    use .min(Integer::compare) to get minimum number
}
```

Output

```
Maximum number in a list: 9
Minimum number in a list: 2
```

1.5 Sort List Of Integers

```
static void sortListOfIntegers(){
    List<Integer> nums = Arrays.asList(1,2,3,4,5);
    int sum = nums.stream()
        .mapToInt(Integer::intValue)
        .sum();
    System.out.println("Sum of All Numbers in a list: " +
        sum);
    use .min(Integer::compare) to get minimum number
}
```

Output

```
Sorted Integers: [1, 2, 3, 5, 9]
```


1.6 Remove Duplicates In A List

```
static void removeDuplicates(){
    List<Integer> nums = Arrays.asList(1,2,2,3,3,4,5);
    List<Integer> result = nums.stream()
        .distinct()
        .toList();
    System.out.println("Remove Duplicates" + result);
}
```

Output

```
Remove Duplicates [1, 2, 3, 4, 5]
```

1.7 Count No Of Elements In List

```
static void countNoOfElementsInList(){
    List<String> list = Arrays.asList("a","b","c","d");
    long count = list.stream().count();
    System.out.println("Count of elements: " + count);
}
```

Output

```
Count of elements: 4
```

1.8 Matching Functions

```
static void matchingFunctions(){
    boolean any = Stream.of(1,2).anyMatch(n -> n>2);
    System.out.println("Any >2: " + any);
    boolean all = Stream.of(2,4,6).allMatch(n -> n%2==0);
    System.out.println("All even: " + all);
    boolean none = Stream.of(1,3,5).noneMatch(n -> n%2==0);
    System.out.println("None even: " + none);
    Stream.of("a","b","c").filter(s->s.startsWith("b"))
        .findFirst().ifPresent(s ->
            System.out.println("First matching: " + s));
}
```

Output

```
Any >2: false
All even: true
None even: true
First matching: b
```

1.9 Count Of Each Character In String

```
static void countOfEachCharacterInString(){
    String input = "Hello World";
    var result = Arrays.stream(input.split(""))
        .filter(s -> !s.isBlank())
```

```
        .collect(Collectors.groupingBy(Function.identity(),
        Collectors.counting()));

        result.forEach((character, count) ->
        System.out.println(character + " : " + count));
    }
```

Output

```
r : 1
d : 1
e : 1
W : 1
H : 1
l : 3
o : 2
```

1.10 Count Of Each Word In String

```
static void countOfEachWordInString(){
    String input = "Hello World Durga Naresh Hello World";
    var result = Arrays.stream(input.split(" "))
        .filter(s -> !s.isBlank())
        .collect(Collectors.groupingBy(Function.identity(),
        Collectors.counting()));

    result.forEach((character, count) ->
    System.out.println(character + " : " + count));
}
```

Output

```
Hello : 2
World : 2
Naresh : 1
Durga : 1
```

1.11 Count Strings Longer Than Five

```
static void countStringsLongerThanFive() {
    List<String> words = Arrays.asList("hello", "world", "
    stream",
    "java", "programming");
    long count = words.stream().filter(w -> w.length() > 5)
        .count();
    System.out.println("Strings with length > 5: " + count)
        ;
}
```

Output

```
Strings with length > 5: 2
```

1.12 Sum Numbers Greater Than Ten

```
static void sumNumbersGreaterThanTen() {  
    List<Integer> numbers = Arrays.asList(5, 15, 20, 8, 12)  
    ;  
    int sum = numbers.stream().filter(n -> n > 10)  
    .mapToInt(Integer::intValue).sum();  
    System.out.println("Sum of numbers > 10: " + sum);  
}
```

Output

```
Sum of numbers > 10: 47
```

1.13 Find First Divisible By Seven

```
static void findFirstDivisibleBySeven() {  
    List<Integer> numbers = Arrays.asList(5, 14, 21, 28, 3)  
    ;  
    numbers.stream()  
    .filter(n -> n % 7 == 0)  
    .findFirst()  
    .ifPresent(n ->  
    System.out.println("First divisible by 7: " + n));  
}
```

Output

```
First divisible by 7: 14
```

1.14 List To Map Word Length

```
static void listToMapWordLength() {  
    List<String> words = Arrays.asList("apple", "banana", "  
    cherry");  
    Map<String, Integer> wordLengthMap = words.stream()  
    .collect(Collectors.toMap(  
    w -> w,  
    String::length  
    ));  
    System.out.println("Word length map: " + wordLengthMap)  
    ;  
}
```

Output

```
Word length map: {banana=6, apple=5, cherry=6}
```

1.15 GroupBy FirstLetter

```
static void groupByFirstLetter() {  
    List<String> names = Arrays.asList("Alice", "Adam", "  
    Bob",
```

```
        "Charlie", "Catherine");
    Map<Character, List<String>> grouped = names.stream()
        .collect(Collectors.groupingBy(n -> n.charAt(0)));
    System.out.println("Grouped by first letter: " +
        grouped);
}
```

Output

```
Grouped by first letter: {A=[Alice, Adam], B=[Bob], C=[Charlie,
    Catherine]}
```

1.16 Partition Even Odd

```
static void partitionEvenOdd() {
    List<Integer> numbers = Arrays.asList(1,2,3,4,5,6);
    Map<Boolean, List<Integer>> partitioned = numbers.
        stream()
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));
    System.out.println(" Even numbers: " + partitioned.get(
        true));
    System.out.println(" Odd numbers: " + partitioned.get(
        false));
}
```

Output

```
Even numbers: [2, 4, 6]
Odd numbers: [1, 3, 5]
```

1.17 Flatten List Of Lists

```
static void partitionEvenOdd() {
static void flattenListOfLists() {
    List<List<Integer>> listOfLists = Arrays.asList(
        Arrays.asList(1,2,3),
        Arrays.asList(4,5,6),
        Arrays.asList(7,8)
    );
    List<Integer> flatList = listOfLists.stream()
        .flatMap(List::stream)
        .toList();
    System.out.println("Flattened list: " + flatList);
}
```

Output

```
Flattened list: [1, 2, 3, 4, 5, 6, 7, 8]
```

1.18 Print First Element

```
static void printFirstElement() {
    List<Integer> nums = Arrays.asList(5, 6, 7);
```

```
        nums.stream().findFirst().ifPresent(n -> System.out.println("
            First element: " + n));
    }
```

Output

```
First element: 5
```

1.19 Optional Find String Starting With A

```
static void optionalFindStringStartingWithA() {
    List<String> names = Arrays.asList("Bob", "Charlie", "
        Alice");
    names.stream()
        .filter(n -> n.startsWith("A"))
        .findAny()
        .map(String::toUpperCase)
        .ifPresentOrElse(
            s -> System.out.println("Found string starting with A:
                " + s),
            () -> System.out.println("No string starts with A")
        );
}
```

Output

```
Found string starting with A: ALICE
```

1.20 Sort Strings By Length Descending and Ascending

```
static void sortStringsByLengthDesc() {
    List<String> names = Arrays.asList("Alice", "Bob", "
        Charlie", "David");
    List<String> sortedDesc = names.stream()
        .sorted((a, b) -> Integer.compare(b.length(), a.length()
            ()))
        .toList();
    System.out.println("Sorted by length descending: " +
        sortedDesc);

    List<Integer> sortedAsc = Stream.of(3,1,2).sorted().
        toList();
    System.out.println("Sorted ascending: " + sortedAsc);

    List<Integer> desc = Stream.of(3,1,2)
        .sorted(Comparator.reverseOrder()).toList();
    System.out.println("Sorted descending: " + desc);
}
```

Output

```
Sorted by length descending: [Charlie, Alice, David, Bob]
Sorted ascending: [1, 2, 3]
Sorted descending: [3, 2, 1]
```

1.21 Collect To Set

```
static void collectToSet() {  
    Set<String> set = new HashSet<>(Arrays.asList("a", "a", "b"));  
    System.out.println("To set: " + set);  
}
```

Output

```
To set: [a, b]
```

1.22 Distinct Numbers

```
static void distinctNumbers() {  
    List<Integer> unique = Stream.of(1,1,2).distinct().toList();  
    System.out.println("Distinct numbers: " + unique);  
}
```

Output

```
Distinct numbers: [1, 2]
```

1.23 Filter And Sort

```
static void filterAndSort() {  
    List<Integer> result = Stream.of(5,2,7,4,8)  
        .filter(n -> n > 4).sorted().toList();  
    System.out.println("Filter >4 & sort: " + result);  
}
```

Output

```
Filter >4 & sort: [5, 7, 8]
```

1.24 Square Each Number

```
static void squareEachNumber() {  
    List<Integer> sq = Stream.of(1,2,3).map(n -> n*n).  
        toList();  
    System.out.println("Squares: " + sq);  
}
```

Output

```
Squares: [1, 4, 9]
```

1.25 Remove Nulls

```
static void removeNulls() {  
    List<String> filtered = Stream.of("a", null, "b").  
        filter(Objects::nonNull)  
        .toList();  
    System.out.println("Remove nulls: " + filtered);  
}
```

Output

```
Remove nulls: [a, b]
```

1.26 Sum Of Evens

```
static void sumOfEvens() {  
    int sum = Stream.of(1,2,3,4).filter(n->n%2==0)  
        .mapToInt(Integer::intValue).sum();  
    System.out.println("Sum of evens: " + sum);  
}
```

Output

```
Sum of evens: 6
```

1.27 Reduce Concatenate Strings And Multiply Integers

```
static void reduceConcatenateStrings() {  
    String joined = Stream.of("a","b","c").reduce("",  
        String::concat);  
    System.out.println("Concatenated: " + joined);  
  
    int product = Stream.of(2,3,4).reduce(1,(a, b) -> a*b);  
    System.out.println("Product: " + product);  
}
```

Output

```
Sum of evens: 6
```

1.28 Join With Comma

```
static void joinWithComma() {  
    String joined = Stream.of("a","b","c").collect(  
        Collectors.joining(","));  
    System.out.println("Joined with comma: " + joined);  
  
    String joined1 = String.join(",", "a", "b", "c");  
    System.out.println("Joined with comma: " + joined1);  
}
```

Output

```
Joined with comma: a,b,c  
Joined with comma: a,b,c
```

1.29 Top3 Highest Value

```
static void top3Highest() {  
    List<Integer> top3 = Stream.of(5,1,9,7)  
        .sorted(Comparator.reverseOrder()).limit(3).toList();  
    System.out.println("Top 3: " + top3);  
}
```

Output

```
Top 3: [9, 7, 5]
```

1.30 Skip First2

```
static void skipFirst2() {  
    List<Integer> skip2 = Stream.of(5,1,9,7)  
        .sorted(Comparator.reverseOrder()).skip(2).toList();  
    System.out.println("Skip 2 3: " + skip2);  
}
```

Output

```
Skip 2: [7, 9]
```

1.31 Debugging using Peek

```
static void peekDebug() {  
    System.out.print("Peek debug: ");  
    Stream.of(1,2,3,4).filter(n->n%2==0)  
        .peek(System.out::print)  
        .toList();  
    System.out.println();  
}
```

Output

1.32 Sum Of Squares Greater Than 10

```
static void sumOfSquaresGreaterThan10() {  
    int sum = Stream.of(3,4,5).map(n->n*n).filter(n->n>10)  
        .mapToInt(i->i).sum();  
    System.out.println("Sum of squares >10: " + sum);  
}
```

Output

1.33 Convert To Array

```
static void convertToArray() {  
    String[] arr = Stream.of("a","b","c").toArray(String  
        []::new);  
    System.out.println("Array: " + Arrays.toString(arr));  
}
```

Output

1.34 Count Frequencies

```
static void countFrequencies() {  
    Map<String,Long> freq = Stream.of("a","a","b")  
        .collect(Collectors.groupingBy(s->s, Collectors.  
            counting()));  
    System.out.println("Frequencies: " + freq);  
}
```

Output

1.35 Unique Sorted Words

```
static void uniqueSortedWords() {  
    List<String> unique = Stream.of("b","a","b").distinct().sorted  
        ()  
        .toList();  
    System.out.println("Unique sorted: " + unique);  
}
```

Output

1.36 Longest String

```
static void longestString() {  
    Stream.of("aa","bbb","c")  
        .max(Comparator.comparingInt(String::length))  
        .ifPresent(s -> System.out.println("Longest: " + s));  
}
```

Output

1.37 Shortest String

```
static void shortestString() {  
    Stream.of("aa","bbb","c")  
        .min(Comparator.comparingInt(String::length))  
        .ifPresent(s -> System.out.println("Shortest: " + s));  
}
```

Output

1.38 Even Numbers Using IntStream

```
static void evenNumbersUsingIntStream() {  
    System.out.print("Even numbers: ");  
    IntStream.range(1,6).filter(n->n%2==0).forEach(n->  
        System.out.print(n+" "));  
}
```

Output

1.39 Generate First NSquares

```
static void generateFirstNSquares() {  
    System.out.print("First 5 squares: ");  
    IntStream.iterate(1,n->n+1).map(n->n*n)  
        .limit(5).forEach(n->System.out.print(n+" "));  
}
```

Output

1.40 Prime Check Using Streams

```
static void primeCheckUsingStreams() {  
    int number = 10;  
    boolean isPrime = IntStream.rangeClosed(2, number-1)  
        .noneMatch(i -> number % i == 0);  
    System.out.println("Is 10 prime? " + isPrime);  
}
```

Output

1.41 Find Palindromes

```
static void findPalindromes() {  
    List<String> pals = Stream.of("aba","abc","aa")  
        .filter(s->new StringBuilder(s).reverse().toString().equals(s))  
        .toList();  
    System.out.println("Palindromes: " + pals);  
}
```

Output

1.42 Distinct Count Of Numbers

```
static void distinctCountOfNumbers() {  
    long count = Stream.of(1,2,2,3).distinct().count();  
    System.out.println("Distinct count: " + count);  
}
```

Output

1.43 Convert List Of Integers To String

```
static void convertListOfIntegersToString() {  
    String result = Stream.of(1,2,3).map(Object::toString)  
        .collect(Collectors.joining("-"));  
    System.out.println("Integers to string: " + result);  
}
```

Output

1.44 Filter By Prefix

```
static void filterByPrefix() {  
    List<String> filtered = Stream.of("apple","bat","ball")  
        .filter(s->s.startsWith("b")).toList();  
    System.out.println("Starts with b: " + filtered);  
}
```

Output

1.45 Remove Negative Numbers

```
static void removeNegativeNumbers() {  
    List<Integer> pos = Stream.of(-1,2,-3,4).filter(n->n  
        >=0).toList();  
    System.out.println("Non-negative: " + pos);  
}
```

Output

1.46 Double The Numbers

```
static void doubleTheNumbers() {  
    List<Integer> doubled = Stream.of(1,2,3).map(n->n*2).  
        toList();  
    System.out.println("Doubled: " + doubled);  
}
```

Output

1.47 Strings To Char Stream

```
static void stringsToCharStream() {  
    List<Character> chars = "abc".chars().mapToObj(c->(char  
        )c).toList();  
    System.out.println("Characters: " + chars);  
}
```

Output

1.48 Average Of Numbers

```
static void averageOfNumbers() {  
    double avg = Stream.of(10,20,30).mapToInt(i->i).average  
        ().orElse(0);  
    System.out.println("Average: " + avg);  
}
```

Output

1.49 Collect To Tree-Set

```
static void collectToTreeSet() {  
    TreeSet<String> treeSet = Arrays.asList("b","a","c").stream().  
        collect(Collectors.toCollection(TreeSet::new));  
    System.out.println("TreeSet: " + treeSet);  
  
    TreeSet<String> treeSet = new TreeSet<>(Arrays.asList("b", "a",  
        "c"));  
    System.out.println("TreeSet: " + treeSet);  
}
```

Output

1.50 Statistics Summary

```
static void statisticsSummary() {  
    IntSummaryStatistics stats = Stream.of(1,2,3,4).  
        mapToInt(i->i)  
        .summaryStatistics();  
    System.out.println("50. Stats: " + stats);  
}
```

Output

1.51 Find-First vs Find-Any in Parallel stream

```
static void findFirstVsFindAny() {
    System.out.println("\nScenario6: findFirst vs findAny");
    ;
    List<String> data = Arrays.asList("A","B","C","D");
    Optional<String> first = data.parallelStream().
        findFirst();
    Optional<String> any = data.parallelStream().findAny();
    System.out.println("findFirst: " + first + ", findAny:
        " + any);
}
```

Output

1.52 Filter out empty Optionals

```
static void filterOptionalEmptyValues() {
    System.out.println("\nScenario7: Filter out empty
        Optionals");
    List<Optional<String>> list = Arrays.asList(Optional.
        empty(), Optional.of("X"), Optional.of("Y"));
    List<String> present = list.stream()
        .filter(Optional::isPresent)
        .map(Optional::get)
        .toList();
    System.out.println("Present values: " + present);
}
```

Output

1.53 Flatten nested List<Optional<List<String>>>;

```
static void flatMapNestedLists() {
    System.out.println("\nScenario8: Flatten nested List<
        Optional<List<String>>>");
    List<Optional<List<String>>> nested = Arrays.asList(
        Optional.of(Arrays.asList("a","b")),
        Optional.empty(),
        Optional.of(Arrays.asList("c","d"))
    );
    List<String> flat = nested.stream()
        .flatMap(Optional::stream)
        .flatMap(List::stream)
        .toList();
    System.out.println("Flattened list: " + flat);
}
```

Output

1.54 or-Else vs orElse-Get performance pitfall

```
static void orElseVsOrElseGet() {
    System.out.println("\nScenario9: orElse vs orElseGet");
    Optional<String> empty = Optional.empty();

    String result1 = empty.orElse(expensiveOperation());
    System.out.println("orElse result: " + result1);

    String result2 = empty.orElseGet(
        StreamOptionalTrickyScenarios::expensiveOperation);
    System.out.println("orElseGet result: " + result2);
}

static String expensiveOperation() {
    System.out.println("Executing expensive operation...");
    return "Default";
}
```

Output

1.55 Short-circuit Optional with Stream Any-Match

```
static void optionalShortCircuit() {
    System.out.println("Short-circuit with Optional and
        Stream.anyMatch");
    List<Optional<Integer>> numbers = Arrays.asList(
        Optional.of(1), Optional.of(2), Optional.empty(),
        Optional.of(4));
    boolean hasEven = numbers.stream()
        .flatMap(Optional::stream)
        .anyMatch(n -> n % 2 == 0);
    System.out.println("Contains even number? " + hasEven);
}
```

Output

Chapter 2

Java 8 Tricky Interview Questions

2.1 Model class used for coming questions

```
static class Person {
    String name; int age; String dept;
    Person(String n,int a,String d){name=n;age=a;
        dept=d;}
    public String toString(){return name+"-"+age+"-
        "+dept;}
}

List<Person> people = Arrays.asList(
    new Person("Alice",30,"HR"),
    new Person("Bob",45,"IT"),
    new Person("Charlie",30,"IT"),
    new Person("David",50,"Finance"),
    new Person("Eve",35,"HR")
);
```

2.2 Group by dept, find oldest in each group

```
static void findOldestinGroup() {
    System.out.println("Oldest person per
        department:");
    Map<String, Optional<Person>> oldestByDept =
        people.stream()
        .collect(Collectors.groupingBy(p->p.dept,
        Collectors.maxBy(Comparator.comparingInt(p->p.
        age))));
    System.out.println(oldestByDept);
}
```

Output

2.3 Partition by age;40 and collect names

```
static void partitionByAge40{
    System.out.println("Partition people by age>40:");
```

```
        Map<Boolean, List<String>> partitioned = people.stream()
            .collect(Collectors.partitioningBy(p->p.age>40,
            Collectors.mapping(p->p.name, Collectors.toList())));
        System.out.println(partitioned);
    }
```

Output

2.4 Lazy evaluation and short-circuiting

```
static void lazyEvaluationAndShortCurcuiting(){
    System.out.println("First person in IT older than 30:");
    ;
    people.stream()
        .filter(p-> {
            System.out.println("Checking "+p.name);
            return p.dept.equals("IT");
        })
        .filter(p->p.age>30)
        .findFirst()
        .ifPresent(System.out::println);
}
```

Output

2.5 Parallel stream with reduce (sum of ages)

```
static void parallelStream{
    System.out.println("Sum of ages using parallel stream:");
    int totalAge = people.parallelStream()
        .map(p->p.age)
        .reduce(0, Integer::sum);
    System.out.println(totalAge);
}
```

Output

2.6 Nested collectors - average age per dept for people older than 30

```
static void parallelStream{
    System.out.println("Average age per dept (age>30):");
    Map<String, Double> avgAge = people.stream()
        .filter(p->p.age>30)
        .collect(Collectors.groupingBy(p->p.dept,
        Collectors.averagingInt(p->p.age)));
    System.out.println(avgAge);
}
```

Output

2.7 Find duplicate ages

```
static void parallelStream{
    System.out.println("Duplicate ages:");
    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = people.stream()
        .map(p->p.age)
        .filter(a->!seen.add(a))
        .collect(Collectors.toSet());
    System.out.println(duplicates);
}
```

Output

2.8 Collecting into a map with merge function (name length)

```
System.out.println("Map dept to total name length:");
Map<String, Integer> nameLengthByDept = people.stream()
    .collect(Collectors.toMap(
        p->p.dept,
        p->p.name.length(),
        Integer::sum // merge function for duplicate keys
    ));
System.out.println(nameLengthByDept);
```

Output

2.9 Find Duplicate Elements in a List

```
static void findDuplicates{
    List<Integer> list = Arrays.asList(1,2,3,4,2,5,3,6);
    Set<Integer> set = new HashSet<>();

    Set<Integer> duplicates = list.stream()
        .filter(n -> !set.add(n))
        .collect(Collectors.toSet());
}
```

Output

2.10 Find First Non-Repeating Character in a String

```
static void nonRepeatingCharacter{
    String str = "javaarticles";

    Character result = str.chars()
        .mapToObj(c -> (char)c)
        .collect(Collectors.groupingBy(c -> c,
            LinkedHashMap::new,
            Collectors.counting()))
}
```

```
        .entrySet()
        .stream()
        .filter(e -> e.getValue() == 1)
        .findFirst()
        .get()
        .getKey();

    System.out.println(result);
}
```

Output

2.11 Find Common Elements Between Two Lists

```
static void nonRepeatingCharacter{
    String str = "javaarticles";

    Character result = str.chars()
        .mapToObj(c -> (char)c)
        .collect(Collectors.groupingBy(c -> c,
            LinkedHashMap::new,
            Collectors.counting()))
        .entrySet()
        .stream()
        .filter(e -> e.getValue() == 1)
        .findFirst()
        .get()
        .getKey();

    System.out.println(result);
}
```

Output

2.12 Convert List to Map

```
static void listToMap{
    String str = "javaarticles";

    Map<Integer, Employee> map =
        employees.stream()
        .collect(Collectors.toMap(
            Employee::getId,
            e -> e
        ));
    System.out.println(map);
}
```

Output

2.13 Find Average Salary of Employees

```
double avgSalary =
    employees.stream()
```

```
.collect(Collectors.averagingDouble(Employee::getSalary));
```

Output

2.14 Count Characters in String

```
Map<Character, Long> map =  
    str.chars()  
    .mapToObj(c -> (char)c)  
    .collect(Collectors.groupingBy(  
        c -> c,  
        Collectors.counting()  
    ));
```

Output

2.15 Sort Employees by Salary

```
List<Employee> sorted =  
    employees.stream()  
    .sorted(Comparator.comparing(Employee::getSalary))  
    .collect(Collectors.toList());
```

Output

2.16 Find Employee with Second Highest Salary

```
Employee emp =  
    employees.stream()  
    .sorted(Comparator.comparing(Employee::getSalary).reversed())  
    .skip(1)  
    .findFirst()  
    .get();
```

Output

2.17 Generate Fibonacci Using Streams

```
Stream.iterate(new int[]{0,1},  
    f -> new int[]{f[1], f[0] + f[1]})  
    .limit(10)  
    .map(f -> f[0])  
    .forEach(System.out::println);
```

Output

2.18 Collecting into a map with merge function (name length)

```
System.out.println("Map dept to total name length:");
Map<String, Integer> nameLengthByDept = people.stream()
    .collect(Collectors.toMap(
        p->p.dept,
        p->p.name.length(),
        Integer::sum // merge function for duplicate keys
    ));
System.out.println(nameLengthByDept);
```

Output

Chapter 3

Map And FlatMap

3.1 Grouping-By with Optional values

```
static void scenario5_groupingWithOptional() {
    System.out.println("\nScenario5: GroupingBy with
        Optional");
    List<User> users = Arrays.asList(
        new User("Alice", Optional.of("alice@example.com")),
        new User("Bob", Optional.empty())
    );

    Map<String, List<User>> grouped = users.stream()
        .collect(Collectors.groupingBy(u -> u.getEmail().orElse(
            "unknown")));
    System.out.println(grouped);
}
```

Output

3.2 Optional with max/min

```
static void scenario4_optionalMaxMin() {
    System.out.println("\nScenario4: Optional with max/min"
    );
    List<Integer> numbers = List.of();
    Optional<Integer> max = numbers.stream().max(Integer::
        compare);
    System.out.println("Max or default (-1): " + max.orElse(
        -1));
}
```

Output

3.3 Optional chaining with possible null objects

```
static void scenario3_optionalChainingWithNull() {
    System.out.println("\nScenario3: Optional chaining with
        possible null");
    List<User> users = Arrays.asList(
        new User("Alice", Optional.of("alice@example.com")),

```

```
        null,
        new User("Bob", Optional.empty())
    );

    List<String> emails = users.stream()
        .map(u -> Optional.ofNullable(u).flatMap(User::getEmail))
        .flatMap(Optional::stream)
        .toList();
    System.out.println("Emails: " + emails);
}
```

Output

3.4 Optional inside Stream

```
static void scenario1_flattenNestedOptionals() {
    System.out.println("Flatten Nested Optionals");
    Optional<Optional<String>> nested = Optional.of(
        Optional.of("Hello"));
    Optional<String> flat = nested.flatMap(o -> o.map(
        String::toUpperCase));
    System.out.println("Result: " + flat); // Optional[
        HELLO]
}
```

Output

3.5 Flatten Nested Optionals

```
static void scenario1_flattenNestedOptionals() {
    System.out.println("Flatten Nested Optionals");
    Optional<Optional<String>> nested = Optional.of(
        Optional.of("Hello"));
    Optional<String> flat = nested.flatMap(o -> o.map(
        String::toUpperCase));
    System.out.println("Result: " + flat); // Optional[
        HELLO]
}
```

Output

3.6 Combined nested collections

```
static void combinedNestedCollections() {
    System.out.println("Combined nested collections");
    List<List<Optional<String>>> data = Arrays.asList(
        Arrays.asList(Optional.of("x"), Optional.empty()),
        Arrays.asList(Optional.of("y"))
    );
}
```

```
List<String> flat = data.stream()
    .flatMap(List::stream)
    .flatMap(Optional::stream)
    .collect(Collectors.toList());
System.out.println(flat); // [x, y]
}
```

Output

3.7 FlatMap multiple streams

```
static void flatMapMultipleStreams() {
    System.out.println("FlatMap multiple streams");
    Stream<String> s1 = Stream.of("a","b");
    Stream<String> s2 = Stream.of("c","d");
    List<String> combined = Stream.of(s1,s2).flatMap(s -> s
        ).collect(Collectors.toList());
    System.out.println(combined); // [a,b,c,d]
}
```

Output

3.8 Empty Optional map vs flatMap

```
static void mapVsFlatMapWithEmptyOptional() {
    System.out.println("Empty Optional map vs flatMap");
    Optional<String> empty = Optional.empty();
    Optional<Optional<Integer>> mapRes = empty.map(s -> Optional.of(
        s.length()));
    Optional<Integer> flatMapRes = empty.flatMap(s -> Optional.of(s
        .length()));
    System.out.println("Map: " + mapRes); // Optional.empty
    System.out.println("FlatMap: " + flatMapRes); // Optional.empty
}
```

Output

3.9 FlatMap Optional containing list

```
static void flatMapOptionalList() {
    System.out.println("FlatMap Optional containing list");
    Optional<List<Integer>> optList = Optional.of(Arrays.
        asList(1,2,3));
    List<Integer> flat = optList.map(List::stream).orElse(
        Stream.empty())
        .collect(Collectors.toList());
    System.out.println(flat); // [1,2,3]
}
```

Output

3.10 FlatMap to Stream (flattened)

```
static void flatMapToStreamNested() {
    System.out.println("FlatMap to Stream (flattened)");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2),
        Arrays.asList(3,4));
    long count = nested.stream().flatMap(List::stream).count();
    System.out.println("Flattened count: " + count); // 4
}
```

Output

3.11 Map to Stream (nested)

```
static void example7_mapToStreamNested() {
    System.out.println("Map to Stream (nested)");
    List<List<Integer>> nested = Arrays.asList(Arrays.asList(1,2),
        Arrays.asList(3,4));
    Stream<Stream<Integer>> s = nested.stream().map(List::stream);
    System.out.println("Stream of streams: " + s.count()); // 2
}
```

Output

3.12 Optional map vs flatMap

```
static void optionalMapVsFlatMap() {
    System.out.println("Optional map vs flatMap");
    Optional<String> opt = Optional.of("hello");

    Optional<Optional<Integer>> mapRes = opt.map(s ->
        Optional.of(s.length()));
    Optional<Integer> flatMapRes = opt.flatMap(s ->
        Optional.of(s.length()));

    System.out.println("Map result: " + mapRes); // Optional[Optional[5]]
    System.out.println("FlatMap result: " + flatMapRes); // Optional[5]
}
```

Output

3.13 FlatMap strings to chars (flattened)

```
static void flatMapStringsToChars() {
    System.out.println("FlatMap strings to chars (flattened)");
    List<String> words = Arrays.asList("hi", "ok");
    List<String> flat = words.stream()
```



```
        .flatMap(w -> Arrays.stream(w.split("")))
        .collect(Collectors.toList());
        System.out.println(flat); // [h,i,o,k]
    }
```

Output

3.14 Map Strings to split arrays → nested

```
static void example4_mapStringsToChars() {
    System.out.println("\nExample4: Map strings to split
        arrays (nested)");
    List<String> words = Arrays.asList("hi","ok");
    List<String[]> mapped = words.stream().map(w -> w.split
        (""))
        .collect(Collectors.toList());
    System.out.println("Mapped list of arrays size: " +
        mapped.size()); // 2
}
```

Output

3.15 Map Strings to split arrays → nested

```
static void example4_mapStringsToChars() {
    System.out.println("\nExample4: Map strings to split
        arrays (nested)");
    List<String> words = Arrays.asList("hi","ok");
    List<String[]> mapped = words.stream().map(w -> w.split
        (""))
        .collect(Collectors.toList());
    System.out.println("Mapped list of arrays size: " +
        mapped.size()); // 2
}
```

Output

3.16 FlatMap flattens nested lists

```
static void example3_flatMapNestedLists() {
    System.out.println("\nExample3: FlatMap flattens nested
        lists");
    List<List<Integer>> nested = Arrays.asList(Arrays.
        asList(1,2), Arrays.asList(3,4));
    List<Integer> flat = nested.stream().flatMap(List::
        stream)
        .collect(Collectors.toList());
    System.out.println(flat); // [1,2,3,4]
}
```

Output

3.17 Simple map

```
static void example1_mapSimple() {
    System.out.println("\nExample1: Simple map");
    List<Integer> nums = Arrays.asList(1,2,3);
    List<Integer> squared = nums.stream().map(n -> n*n).
        collect(Collectors.toList());
    System.out.println(squared); // [1, 4, 9]
}
```

Output

3.18 Optional stream flatten

```
static void optionalFlatMap8() {
    System.out.println("\nOptionalFlatMap8: Optional stream
        flatten");
    Optional<List<Integer>> optList = Optional.of(Arrays.
        asList(1,2,3));
    long start = System.nanoTime();
    List<Integer> flat = optList.map(List::stream).orElse(
        Stream.empty())
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flattened Optional stream: " + flat
        + ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.19 Optional filter + flatMap

```
static void optionalFlatMap7() {
    System.out.println("\nOptionalFlatMap7: Optional filter
        + flatMap");
    Optional<String> opt = Optional.of("hello");
    long start = System.nanoTime();
    Optional<String> res = opt.filter(s->s.startsWith("h"))
        .flatMap(s->Optional.of(s.toUpperCase()));
    long end = System.nanoTime();
    System.out.println("Filtered + flatMap: " + res.get() +
        ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.20 Optional map chaining

```
static void optionalFlatMap6() {
    System.out.println("\nOptionalFlatMap6: Optional map
        chaining");
    Optional<String> opt = Optional.of("java");
    long start = System.nanoTime();
    Optional<Integer> len = opt.map(String::toUpperCase).
        map(String::length);
    long end = System.nanoTime();
    System.out.println("Length after map chaining: " + len.
        get() + ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.21 Optional of Optional empty

```
static void optionalFlatMap5() {
    System.out.println("\nOptionalFlatMap5: Optional of
        Optional empty");
    Optional<Optional<String>> opt = Optional.of(Optional.
        empty());
    long start = System.nanoTime();
    Optional<String> flat = opt.flatMap(o->o);
    long end = System.nanoTime();
    System.out.println("Flattened Optional: " + flat.orElse
        ("empty") + ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.22 Optional map vs flatMap with list

```
static void optionalFlatMap4() {
    System.out.println("\nOptionalFlatMap4: Optional map vs
        flatMap with list");
    Optional<List<String>> opt = Optional.of(Arrays.asList(
        "a","b"));
    long start = System.nanoTime();
    List<String> resMap = opt.map(l->l).orElse(Collections.
        emptyList());
    List<String> resFlatMap = opt.flatMap(l->Optional.of(l)
        )
        .orElse(Collections.emptyList());
    long end = System.nanoTime();
    System.out.println("Map: " + resMap + ", FlatMap: " +
        resFlatMap +
        ", Time(ms): " + (end-start)/1_000_000);
}
```

Output

3.23 Optional empty handling

```
static void optionalFlatMap3() {
    System.out.println("\nOptionalFlatMap3: Optional empty
        handling");
    Optional<String> empty = Optional.empty();
    long start = System.nanoTime();
    String result = empty.flatMap(s->Optional.of(s.
        toUpperCase())).orElse("default");
    long end = System.nanoTime();
    System.out.println("Result for empty Optional: " +
        result + ", Time(ms): " +
        (end-start)/1_000_000);
}
```

Output

3.24 Nested Optionals flatten

```
static void optionalFlatMap2() {
    System.out.println("\nOptionalFlatMap2: Nested
        Optionals flatten");
    Optional<Optional<String>> nested = Optional.of(
        Optional.of("hello"));
    long start = System.nanoTime();
    Optional<String> flat = nested.flatMap(o->o);
    long end = System.nanoTime();
    System.out.println("Flattened nested Optional: " + flat
        .get() + ", Time(ms): " +
        (end-start)/1_000_000);
}
```

Output

3.25 Flatten Optional of List

```
static void optionalFlatMap1() {
    System.out.println("\nOptionalFlatMap1: Flatten
        Optional of List");
    Optional<List<Integer>> optList = Optional.of(Arrays.
        asList(1,2,3,4,5));
    long start = System.nanoTime();
    List<Integer> flat = optList.map(List::stream).orElse(
        Stream.empty())
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flattened Optional: " + flat + ",
        Time(ms): "
```

```
        + (end-start)/1_000_000);
    }
```

Output

3.26 Combined nested collections + Optional flatten

```
static void mapVsFlatMap12() {
    System.out.println("Combined nested collections +
        Optional flatten");
    List<List<Optional<String>>> data = Arrays.asList(
        Arrays.asList(Optional.of("x"), Optional.empty()),
        Arrays.asList(Optional.of("y"))
    );
    long start = System.nanoTime();
    List<String> flat = data.stream().flatMap(List::stream)
        .flatMap(Optional::stream)
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flattened result: " + flat + ",
        Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.27 FlatMap multiple streams

```
static void mapVsFlatMap11() {
    System.out.println("FlatMap multiple streams");
    Stream<String> s1 = Stream.of("a","b");
    Stream<String> s2 = Stream.of("c","d");
    long start = System.nanoTime();
    List<String> combined = Stream.of(s1,s2).flatMap(s->s).
        collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Combined: " + combined + ", Time(ms)
        ): " + (end-start)/1_000_000);
}
```

Output

3.28 Empty Optional map vs flatMap

```
static void mapVsFlatMap10() {
    System.out.println("Empty Optional map vs flatMap");
    Optional<String> empty = Optional.empty();
    long start = System.nanoTime();
    Optional<Optional<Integer>> mapRes = empty.map(s->
        Optional.of(s.length()));
}
```

```
Optional<Integer> flatMapRes = empty.flatMap(s->
    Optional.of(s.length()));
long end = System.nanoTime();
System.out.println("Map: " + mapRes + ", FlatMap: " +
    flatMapRes + ", Time(ms): "
    + (end-start)/1_000_000);
}
```

Output

3.29 Flatten Optional containing list

```
static void mapVsFlatMap9() {
    System.out.println("Flatten Optional containing list");
    Optional<List<Integer>> optList = Optional.of(Arrays.asList
        (1,2,3,4,5));
    long start = System.nanoTime();
    List<Integer> flat = optList.map(List::stream).orElse(Stream.
        empty()).toList();
    long end = System.nanoTime();
    System.out.println("Flattened Optional: " + flat + ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.30 Stream of streams flattened

```
static void mapVsFlatMap8() {
    System.out.println("Stream of streams flattened");
    List<List<Integer>> nested = Arrays.asList(Arrays.
        asList(1,2), Arrays.asList(3,4));
    long start = System.nanoTime();
    long count = nested.stream().flatMap(List::stream).
        count();
    long end = System.nanoTime();
    System.out.println("Flattened count: " + count + ",
        Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.31 Stream of streams with map

```
static void mapVsFlatMap7() {
    System.out.println("Stream of streams with map");
    List<List<Integer>> nested = Arrays.asList(Arrays.
        asList(1,2), Arrays.asList(3,4));
```

```
        long start = System.nanoTime();
        Stream<Stream<Integer>> s = nested.stream().map(List::
            stream);
        long end = System.nanoTime();
        System.out.println("Stream of streams size: 2, Time(ms)
            : " + (end-start)/1_000_000);
    }
```

Output

3.32 Simple map (square numbers)

```
static void mapVsFlatMap1() {
    System.out.println("Simple map (square numbers)");
    List<Integer> nums = IntStream.rangeClosed(1, 1_000_000
        ).boxed()
        .collect(Collectors.toList());
    long start = System.nanoTime();
    List<Integer> squared = nums.stream().map(n -> n * n).
        limit(10)
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("First 10 squared: " + squared + ",
        Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.33 Nested lists with map (nested)

```
static void mapVsFlatMap2() {
    System.out.println("Nested lists with map (nested)");
    List<List<Integer>> nested = new ArrayList<>();
    for(int i=0; i<1000; i++) nested.add(IntStream.range(i
        *1000, (i+1)*1000)
        .boxed().collect(Collectors.toList()));
    long start = System.nanoTime();
    List<Stream<Integer>> mapped = nested.stream().map(List
        ::stream)
        .limit(2).collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Mapped (Stream<Stream>) size: " +
        mapped.size() + ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.34 Nested lists with flatMap (flattened)

```
static void mapVsFlatMap3() {
    System.out.println("Nested lists with flatMap (
        flattened)");
    List<List<Integer>> nested = new ArrayList<>();
    for(int i=0;i<1000;i++) nested.add(IntStream.range(i
        *1000,(i+1)*1000)
        .boxed().collect(Collectors.toList()));
    long start = System.nanoTime();
    List<Integer> flat = nested.stream().flatMap(List::
        stream).limit(10)
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flat list first 10: " + flat + ",
        Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.35 Map strings to split arrays (nested)

```
static void mapVsFlatMap4() {
    System.out.println("Map strings to split arrays (nested
        )");
    List<String> words = Arrays.asList("hi","ok","java");
    long start = System.nanoTime();
    List<String[]> mapped = words.stream().map(w->w.split("
        "))
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Mapped list of arrays size: " +
        mapped.size() + ", Time(ms): "
        + (end-start)/1_000_000);
}
```

Output

3.36 FlatMap strings to chars (flattened)

```
static void mapVsFlatMap5() {
    System.out.println("FlatMap strings to chars (flattened
        )");
    List<String> words = Arrays.asList("hi","ok","java");
    long start = System.nanoTime();
    List<String> flat = words.stream().flatMap(w->Arrays.
        stream(w.split("")))
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Flattened chars: " + flat + ", Time
        (ms): "
        + (end-start)/1_000_000);
}
```



```
}
```

Output

3.37 Optional map vs flatMap

```
static void mapVsFlatMap6() {
    System.out.println("Optional map vs flatMap");
    Optional<String> opt = Optional.of("hello");
    long start = System.nanoTime();
    Optional<Optional<Integer>> mapRes = opt.map(s->
        Optional.of(s.length()));
    Optional<Integer> flatMapRes = opt.flatMap(s->Optional.
        of(s.length()));
    long end = System.nanoTime();
    System.out.println("Map result: " + mapRes + ", FlatMap
        result: " + flatMapRes +
        ", Time(ms): " + (end-start)/1_000_000);
}
```

Output

3.38 Concurrent collectors

```
static void scenario7_collectorsConcurrent() {
    System.out.println("\nScenario7: Concurrent collectors
        with parallel stream");
    List<Integer> nums = IntStream.rangeClosed(1, 1_000_000
        ).boxed().collect(Collectors.toList());
    Map<Integer, Long> grouped = nums.parallelStream()
        .collect(Collectors.groupingByConcurrent(i -> i%10,
            Collectors.counting()));
    System.out.println("Grouped count (mod 10): " + grouped
        );
}
```

Output

3.39 Concurrent collectors

```
static void scenario7_collectorsConcurrent() {
    System.out.println("\nScenario7: Concurrent collectors
        with parallel stream");
    List<Integer> nums = IntStream.rangeClosed(1, 1_000_000
        ).boxed().collect(Collectors.toList());
    Map<Integer, Long> grouped = nums.parallelStream()
        .collect(Collectors.groupingByConcurrent(i -> i%10,
            Collectors.counting()));
    System.out.println("Grouped count (mod 10): " + grouped
        );
}
```

Output

3.40 Parallel Map-Reduce large dataset

```
static void scenario6_parallelMapReduceLarge() {
    System.out.println("\nScenario6: Parallel map-reduce
        large dataset");
    List<Integer> nums = IntStream.rangeClosed(1, 10
        _000_000).boxed().collect(Collectors.toList());
    long start = System.nanoTime();
    long sum = nums.parallelStream().mapToLong(i -> i).sum
        ();
    long end = System.nanoTime();
    System.out.println("Sum: " + sum + ", Time(ms): " + (
        end-start)/1_000_000);
}
```

Output

3.41 Short-circuiting parallel

```
static void scenario5_shortCircuitParallel() {
    System.out.println("\nScenario5: Short-circuiting in
        parallel");
    List<Integer> nums = IntStream.rangeClosed(1, 1000).
        boxed().collect(Collectors.toList());
    boolean found = nums.parallelStream().anyMatch(i -> i >
        500);
    System.out.println("Found number > 500? " + found);
}
```

Output

3.42 Custom ForkJoinPool

```
static void scenario4_customForkJoinPool() {
    System.out.println("\nScenario4: Custom ForkJoinPool");
    ForkJoinPool pool = new ForkJoinPool(4);
    List<Integer> list = IntStream.rangeClosed(1, 1_000_000
        ).boxed().collect(Collectors.toList());
    try {
        int sum = pool.submit(() -> list.parallelStream
            ().reduce(0, Integer::sum)).get();
        System.out.println("Sum with custom pool: " +
            sum);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Output

3.43 Optional in parallel stream

```
static void scenario3_optionalParallelStream() {
    System.out.println("\nScenario3: Optional + Parallel
                        Stream");
    List<Optional<Integer>> numbers = Arrays.asList(
        Optional.of(1), Optional.empty(),
        Optional.of(4));
    boolean hasEven = numbers.parallelStream()
        .flatMap(Optional::stream)
        .anyMatch(n -> n % 2 == 0);
    System.out.println("Contains even? " + hasEven);
}
```

Output

3.44 Parallel ordering

```
static void scenario2_parallelOrdering() {
    System.out.println("\nScenario2: Parallel ordering");
    List<String> data = Arrays.asList("A", "B", "C", "D");
    System.out.print("forEach unordered: ");
    data.parallelStream().forEach(System.out::print);

    System.out.print("\nforEachOrdered: ");
    data.parallelStream().forEachOrdered(System.out::print);
    ;
    System.out.println();
}
```

Output

3.45 Shared mutable state in parallel stream

```
static void scenario1_sharedMutableState() {
    System.out.println("\nScenario1: Shared mutable state");
    ;
    List<Integer> nums = IntStream.rangeClosed(1, 1000)
        .boxed().collect(Collectors.toList());
    List<Integer> unsafe = new ArrayList<>();
    nums.parallelStream().forEach(unsafe::add);
    System.out.println("Unsafe size (may be <1000): " +
        unsafe.size());

    List<Integer> safe = nums.parallelStream().collect(
        Collectors.toList());
    System.out.println("Safe size: " + safe.size());
}
```

Output

3.46 Parallel vs sequential benchmark

```
static void exercise12_parallelVsSequentialBenchmark() {
    System.out.println("\nExercise12: Parallel vs
        sequential benchmark");
    List<Integer> list = IntStream.rangeClosed(1,10_000_000)
        .boxed()
        .collect(Collectors.toList());

    long t1 = System.nanoTime();
    long sumSeq = list.stream().mapToLong(i -> i).sum();
    long t2 = System.nanoTime();
    System.out.println("Sequential sum: " + sumSeq + ",
        Time: " + (t2-t1)/1_000_000 + " ms");

    long t3 = System.nanoTime();
    long sumPar = list.parallelStream().mapToLong(i -> i).
        sum();
    long t4 = System.nanoTime();
    System.out.println("Parallel sum: " + sumPar + ", Time:
        " + (t4-t3)/1_000_000 + " ms");
}
```

Output

3.47 Find top N elements efficiently

```
static void exercise11_findTopN() {
    System.out.println("\nExercise11: Top N elements
        efficiently");
    List<Integer> list = new Random().ints(1_000_000,1,1
        _000_000).boxed()
        .collect(Collectors.toList());
    List<Integer> top10 = list.stream()
        .sorted(Comparator.reverseOrder())
        .limit(10)
        .collect(Collectors.toList());
    System.out.println("Top 10 elements: " + top10.subList
        (0,5) + " ...");
}
```

Output

3.48 Sequential vs Parallel Streams

```
static void exercise1_sequentialVsParallel() {
    System.out.println("\nExercise1: Sequential vs Parallel
        Streams");
    List<Integer> list = IntStream.rangeClosed(1, 1_000_000)
        .boxed().collect(Collectors.toList());

    long t1 = System.nanoTime();
    long sumSeq = list.stream().mapToLong(i -> i).sum();
```

```
        long t2 = System.nanoTime();
        System.out.println("Sequential sum: " + sumSeq + ",
            Time: " + (t2-t1)/1_000_000 + " ms");

        long t3 = System.nanoTime();
        long sumPar = list.parallelStream().mapToLong(i -> i).
            sum();
        long t4 = System.nanoTime();
        System.out.println("Parallel sum: " + sumPar + ", Time:
            " + (t4-t3)/1_000_000 + " ms");
    }
```

Output

3.49 Lazy evaluation example

```
static void exercise2_lazyEvaluation() {
    System.out.println("\nExercise2: Lazy evaluation");
    List<Integer> list = Arrays.asList
        (1,2,3,4,5,6,7,8,9,10);
    int firstEven = list.stream()
        .filter(i -> {
            System.out.println("Filtering " + i);
            return i % 2 == 0;
        })
        .map(i -> {
            System.out.println("Mapping " + i);
            return i * 2;
        })
        .findFirst()
        .orElse(-1);
    System.out.println("First even doubled: " + firstEven);
}
```

Output

3.50 Primitive Streams avoid boxing/unboxing

```
static void exercise3_primitiveStreams() {
    System.out.println("\nExercise3: Primitive Streams");
    int sum = IntStream.rangeClosed(1,1_000_000).map(i -> i
        *i).sum();
    System.out.println("Sum of squares (primitive stream):
        " + sum);
}
```

Output

3.51 Short-circuit filter with limit

```
static void exercise4_shortCircuitFilter() {
    System.out.println("\nExercise4: Short-circuit with
        limit");
    List<Integer> list = IntStream.rangeClosed(1, 100).
        boxed().collect(Collectors.toList());
    List<Integer> limited = list.stream()
        .filter(i -> i % 3 == 0)
        .limit(5)
        .collect(Collectors.toList());
    System.out.println("First 5 divisible by 3: " + limited
        );
}
```

Output

3.52 Short-circuit filter with limit

```
static void exercise4_shortCircuitFilter() {
    System.out.println("\nExercise4: Short-circuit with
        limit");
    List<Integer> list = IntStream.rangeClosed(1, 100).
        boxed().collect(Collectors.toList());
    List<Integer> limited = list.stream()
        .filter(i -> i % 3 == 0)
        .limit(5)
        .collect(Collectors.toList());
    System.out.println("First 5 divisible by 3: " + limited
        );
}
```

Output

3.53 Parallel stream with shared mutable state (pitfall)

```
static void exercise5_parallelSharedState() {
    System.out.println("\nExercise5: Parallel stream shared
        state pitfall");
    List<Integer> nums = IntStream.rangeClosed(1,1000).
        boxed().collect(Collectors.toList());
    List<Integer> results = new ArrayList<>();
    nums.parallelStream().forEach(results::add); // not
        thread-safe
    System.out.println("Size (may be incorrect): " +
        results.size());

    // Correct
    List<Integer> safeResults = nums.parallelStream().
        collect(Collectors.toList());
    System.out.println("Safe results size: " + safeResults.
        size());
}
```

Output

3.54 Sorting large streams

```
static void exercise6_sortingLargeStreams() {
    System.out.println("\nExercise6: Sorting large stream")
    ;
    List<Integer> list = new Random().ints(1_000_000,1,1
        _000_000).boxed().collect(Collectors.toList());
    long t1 = System.nanoTime();
    list.stream().sorted().limit(10).count(); // avoid
        unnecessary full sort
    long t2 = System.nanoTime();
    System.out.println("Sorted top 10 in time: " + (t2-t1)
        /1_000_000 + " ms");
}
```

Output

3.55 Collectors efficiency

```
static void exercise7_largeCollectors() {
    System.out.println("\nExercise7: Collector efficiency")
    ;
    List<String> data = IntStream.range(1,100_000).mapToObj
        (String::valueOf)
        .collect(Collectors.toList());

    long t1 = System.nanoTime();
    String joined = data.stream().collect(Collectors.
        joining(", "));
    long t2 = System.nanoTime();
    System.out.println("Joining 100k strings took: " + (t2-
        t1)/1_000_000 + " ms");
}
```

Output

3.56 Limit early

```
static void exercise8_limitEarly() {
    System.out.println("\nExercise8: Limit early for
        performance");
    List<Integer> list = IntStream.rangeClosed(1,1_000_000)
        .boxed()
        .collect(Collectors.toList());
    long t1 = System.nanoTime();
    List<Integer> top5 = list.stream().limit(5).collect(
        Collectors.toList());
    long t2 = System.nanoTime();
    System.out.println("Top 5 first elements took: " + (t2-
        t1)/1_000_000 + " ms");
}
```

```
}
```

Output

3.57 Map/reduce optimization

```
static void exercise9_mapReduceOptimization() {
    System.out.println("\nExercise9: Map/reduce
        optimization");
    long sumNaive = IntStream.rangeClosed(1,1_000_000)
        .boxed()
        .map(i -> i*i)
        .reduce(0,Integer::sum);
    System.out.println("Sum squares naive (boxed): " +
        sumNaive);

    long sumOptimized = IntStream.rangeClosed(1,1_000_000)
        .map(i -> i*i)
        .sum();
    System.out.println("Sum squares optimized (primitive):
        " + sumOptimized);
}
```

Output

3.58 Boxing/unboxing overhead

```
static void exercise10_boxingUnboxing() {
    System.out.println("\nExercise10: Boxing/unboxing
        overhead");
    List<Integer> list = IntStream.rangeClosed(1,1_000_000)
        .boxed().collect(Collectors.toList());

    long t1 = System.nanoTime();
    long sum = list.stream().mapToInt(Integer::intValue).
        sum();
    long t2 = System.nanoTime();
    System.out.println("Sum with unboxing: " + sum + ",
        Time: " + (t2-t1)/1_000_000 + " ms");
}
```

Output

Chapter 4

Employee Based Programs

4.1 Model class used for coming questions

```
static class Employee {
    String name, dept, gender;
    int salary;

    Employee(String name, String dept, int salary, String
        gender) {
        this.name = name; this.dept = dept; this.salary
            = salary; this.gender = gender;
    }

    public String getName() { return name; }
    public String getDept() { return dept; }
    public int getSalary() { return salary; }
    public String getGender() { return gender; }

    @Override
    public String toString() {
        return name + "(" + salary + ")";
    }
}
```

4.2 Group by dept → gender → top 2 salaries

```
static void problem1() {
    List<Employee> employees = Arrays.asList(
        new Employee("Alice", "IT", 1000, "F"),
        new Employee("Bob", "IT", 1200, "M"),
        new Employee("Carol", "IT", 1500, "F"),
        new Employee("David", "HR", 1300, "M"),
        new Employee("Eve", "HR", 1250, "F")
    );

    Map<String, Map<String, List<Employee>>> result =
        employees.stream()
            .collect(Collectors.groupingBy(Employee::getDept,
                Collectors.groupingBy(Employee::getGender,
                    Collectors.collectingAndThen(Collectors.toList(),
                        list -> list.stream())
```

```

        .sorted(Comparator.comparing
        (Employee::getSalary).reversed())
        .limit(2)
        .collect(Collectors.toList())
        ))));

    System.out.println("Problem1 Result: " + result);
}

```

Output

4.3 Median salary per dept

```

static void problem2() {
    List<Employee> employees = Arrays.asList(
        new Employee("Alice", "IT", 1000, "F"),
        new Employee("Bob", "IT", 1200, "M"),
        new Employee("Carol", "IT", 1500, "F"),
        new Employee("David", "HR", 1300, "M"),
        new Employee("Eve", "HR", 1250, "F")
    );

    Map<String, Double> medianSalary =
        employees.stream()
        .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.collectingAndThen(Collectors.toList(), list
        -> {
            List<Integer> salaries = list.stream()
            .map(Employee::getSalary)
            .sorted()
            .collect(Collectors.toList());
            int n = salaries.size();
            if (n % 2 == 0) return (salaries.get(n/2-1)+
            salaries.get(n/2))/2.0;
            else return (double)salaries.get(n/2);
        })));

    System.out.println("Problem2 Median Salary: " +
        medianSalary);
}

```

Output

4.4 Flatten nested list of Optional<Employee>

```

static void problem3() {
    List<List<Optional<Employee>>> nested = Arrays.asList(
        Arrays.asList(Optional.of(new Employee("Alice", "IT",
        1000, "F")), Optional.empty()),
        Arrays.asList(Optional.of(new Employee("Bob", "IT", 1200,
        "M"))))
    );
}

```

```
List<Employee> flat = nested.stream()
    .flatMap(List::stream)
    .filter(Optional::isPresent)
    .map(Optional::get)
    .collect(Collectors.toList());

System.out.println("Problem3 Flattened: " + flat);
}
```

Output

4.5 Second highest salary

```
static void problem4() {
    List<Employee> employees = Arrays.asList(
        new Employee("Alice", "IT", 1000, "F"),
        new Employee("Bob", "IT", 1200, "M"),
        new Employee("Carol", "IT", 1500, "F")
    );

    Integer secondHighest = employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst()
        .orElse(null);

    System.out.println("Problem4 Second Highest Salary: " +
        secondHighest);
}
```

Output

4.6 Top 3 salaries per dept

```
static void problem5() {  
    List<Employee> employees = Arrays.asList(  
        new Employee("Alice", "IT", 1000, "F"),  
        new Employee("Bob", "IT", 1200, "M"),  
        new Employee("Carol", "IT", 1500, "F"),  
        new Employee("David", "HR", 1300, "M"),  
        new Employee("Eve", "HR", 1250, "F")  
    );  
  
    Map<String, List<Integer>> top3ByDept = employees.  
        stream()  
        .collect(Collectors.groupingBy(Employee::getDept,  
            Collectors.collectingAndThen(Collectors.toList(),  
                list -> list.stream()  
            ).map(Employee::getSalary)  
            .sorted(Comparator.reverseOrder())  
            .limit(3)
```

```

        .collect(Collectors.toList())
    ));

    System.out.println("Problem5 Top 3 Salaries: " +
        top3ByDept);
}

```

Output

4.7 Flatten List<List<String>> and remove duplicates

```

static void problem6() {
    List<List<String>> nested = Arrays.asList(
        Arrays.asList("Java", "Python"),
        Arrays.asList("Python", "C++"),
        Arrays.asList("Go", "Java")
    );

    List<String> flat = nested.stream()
        .flatMap(List::stream)
        .distinct()
        .collect(Collectors.toList());

    System.out.println("Problem6 Unique Languages: " + flat
    );
}

```

Output

4.8 Count word frequency

```

static void problem7() {
    String text = "java python java c++ python java go";
    Map<String, Long> freq = Arrays.stream(text.split("\\s+"))
        .collect(Collectors.groupingBy(s -> s, Collectors.
            counting()));

    System.out.println("Problem7 Word Frequency: " + freq);
}

```

Output

4.9 Find longest string starting with vowel

```

static void problem8() {
    List<String> words = Arrays.asList("apple", "banana", "
        orange", "umbrella", "grape");
    Optional<String> longestVowel = words.stream()
        .filter(s -> s.matches("^[AEIOUaeiou].*"))

```

```
        .max(Comparator.comparingInt(String::length));
        System.out.println("Problem8 Longest Vowel String: " +
            longestVowel.orElse(""));
    }
```

Output

4.10 Flatten nested map to entries

```
static void problem9() {
    Map<String, Map<String,Integer>> nested = new HashMap
        <>();
    nested.put("Dept1", Map.of("Alice",1000,"Bob",1200));
    nested.put("Dept2", Map.of("Carol",1500,"David",1300));

    List<String> flat = nested.entrySet().stream()
        .flatMap(e -> e.getValue().entrySet().stream()
            .map(inner -> e.getKey()+"."+inner.getKey()+"="+inner.
                getValue()))
        .collect(Collectors.toList());

    System.out.println("Problem9 Flattened Map Entries: " +
        flat);
}
```

Output

4.11 Top N frequent elements

```
static void problem10() {
    List<String> items = Arrays.asList("apple","banana","
        apple","orange","banana","apple");
    int N = 2;
    List<String> topN = items.stream()
        .collect(Collectors.groupingBy(s -> s, Collectors.
            counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<String,Long>comparingByValue().
            reversed())
        .limit(N)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());

    System.out.println("Problem10 Top N Items: " + topN);
}
```

Output

4.12 Employees above 75th percentile

```
static void problem11() {
    List<Employee> employees = Arrays.asList(
        new Employee("Alice", "IT", 1000, "F"),
        new Employee("Bob", "IT", 1200, "M"),
        new Employee("Carol", "IT", 1500, "F"),
        new Employee("David", "IT", 1300, "M")
    );
    List<Integer> salaries = employees.stream().map(
        Employee::getSalary).sorted()
        .collect(Collectors.toList());
    int n = salaries.size();
    double p75 = salaries.get((int) Math.ceil(0.75*n)-1);
    List<Employee> above75 = employees.stream().filter(e ->
        e.getSalary() >= p75)
        .collect(Collectors.toList());

    System.out.println("Problem11 Above 75th percentile: "
        + above75);
}
```

Output

4.13 Optional chaining: nested field length

```
static void problem12() {
    Optional<Employee> emp = Optional.of(new Employee("
        Alice", "IT", 1000, "F"));
    int len = emp.map(Employee::getName).map(String::length)
        .orElse(0);
    System.out.println("Problem12 Name length: " + len);
}
```

Output

4.14 First non-empty Optional

```
static void problem13() {
    Optional<String> first = (Optional<String>) Stream.of(
        Optional.empty(), Optional.of("Hello"), Optional.of(
            "World"))
        .filter(Optional::isPresent)
        .map(Optional::get)
        .findFirst();
    System.out.println("Problem13 First non-empty: " +
        first.orElse(""));
}
```

Output

4.15 Combine multiple Optionals

```
static void problem14() {
    Optional<String> a = Optional.of("Java");
    Optional<String> b = Optional.of("Stream");
    Optional<String> combined = a.flatMap(x -> b.map(y -> x
        + " " + y));
    System.out.println("Problem14 Combined: " + combined.
        orElse(""));
}
```

Output

4.16 Parallel sum of squares

```
static void problem15() {
    List<Integer> nums = IntStream.rangeClosed(1,1000).
        boxed().collect(Collectors.toList());
    int sum = nums.parallelStream().mapToInt(i -> i*i).sum
        ();
    System.out.println("Problem15 Sum of squares: " + sum);
}
```

Output

4.17 Parallel word count ignoring stop words

```
static void problem16() {
    List<String> text = Arrays.asList("java","python","java",
        "","c++","python","java","the","and");
    Set<String> stop = Set.of("the","and");
    Map<String, Long> freq = text.parallelStream()
        .filter(w -> !stop.contains(w))
        .collect(Collectors.groupingBy(w -> w, Collectors.
            counting()));
    System.out.println("Problem16 Word count: " + freq);
}
```

Output

4.18 Update ConcurrentHashMap with merge

```
static void problem17() {
    List<String> events = Arrays.asList("login","logout","
        login","login");
    Map<String,Integer> counts = new ConcurrentHashMap<>();
    events.parallelStream().forEach(e -> counts.merge(e,1,
        Integer::sum));
    System.out.println("Problem17 Event counts: " + counts)
        ;
}
```

Output

4.19 Top 10 frequent items from multiple lists

```
static void problem18() {
    List<String> list1 = Arrays.asList("a","b","a");
    List<String> list2 = Arrays.asList("b","c","a");
    List<String> merged = Stream.concat(list1.stream(),
        list2.stream()).collect(Collectors.toList());
    List<String> top = merged.stream()
        .collect(Collectors.groupingBy(s->s,Collectors.counting
            ()))
        .entrySet().stream()
        .sorted(Map.Entry.<String,Long>comparingByValue().
            reversed())
        .limit(10)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());
    System.out.println("Problem18 Top items: " + top);
}
```

Output

4.20 Flatten tree-like structure

```
static void problem19() {
    Employee alice = new Employee("Alice","IT",1000,"F");
    Employee bob = new Employee("Bob","IT",1200,"M");
    Map<Employee,List<Employee>> tree = Map.of(alice, List.
        of(bob));
    List<Employee> all = tree.entrySet().stream()
        .flatMap(e -> Stream.concat(Stream.of(e.getKey()), e.
            getValue().stream()))
        .collect(Collectors.toList());
    System.out.println("Problem19 Flattened tree: " + all);
}
```

Output

4.21 Compute cumulative sum

```
static void problem20() {
    List<Integer> nums = Arrays.asList(1,2,3,4,5);
    List<Integer> cumSum = new ArrayList<>();
    nums.stream().reduce(0,(a,b)->{ int s=a+b; cumSum.add(s
        ); return s; });
    System.out.println("Problem20 Cumulative sum: " +
        cumSum);
}
```

Output

4.22 Flatten nested List<Optional<List<String>>> to distinct sorted strings

```
static void problem21() {
    List<Optional<List<String>>> nested = Arrays.asList(
        Optional.of(Arrays.asList("a", "b")),
        Optional.of(Arrays.asList("b", "c")),
        Optional.empty()
    );
    List<String> flat = nested.stream()
        .filter(Optional::isPresent)
        .map(Optional::get)
        .flatMap(List::stream)
        .distinct()
        .sorted()
        .collect(Collectors.toList());
    System.out.println("Problem21 Flattened strings: " +
        flat);
}
```

Output

4.23 Fibonacci using Stream.iterate

```
static void problem23() {
    List<Integer> fib = Stream.iterate(new int[]{0,1}, a ->
        new int[]{a[1], a[0]+a[1]})
        .limit(10)
        .map(a -> a[0])
        .collect(Collectors.toList());
    System.out.println("Problem23 Fibonacci: " + fib);
}
```

Output

4.24 Pagination with skip/limit

```
static void problem24() {
    List<Integer> nums = IntStream.rangeClosed(1,100).boxed()
        .collect(Collectors.toList());
    int page = 2, size = 10;
    List<Integer> pageItems = nums.stream().skip((page-1)*
        size).limit(size)
        .collect(Collectors.toList());
    System.out.println("Problem24 Page items: " + pageItems
        );
}
```

Output

4.25 Prime numbers using streams

```
static void problem25() {
    List<Integer> primes = IntStream.rangeClosed(2,50)
        .filter(SeniorJava8StreamProblems::isPrime)
        .boxed()
        .collect(Collectors.toList());
    System.out.println("Problem25 Primes: " + primes);
}

static boolean isPrime(int n) {
    return n>1 && IntStream.rangeClosed(2,(int)Math.sqrt(n))
        .allMatch(i->n%i!=0);
}
```

Output

4.26 Short-circuit operation (anyMatch)

```
static void problem22() {
    List<Integer> nums = Arrays.asList(1,3,5,6,7);
    boolean hasEven = nums.stream().anyMatch(n -> n%2==0);
    System.out.println("Problem22 Has even? " + hasEven);
}
```

Output

Chapter 5

Advanced Stream Multithreading Exercises

5.1 Parallel ordering

```
static void scenario2_parallelOrdering() {
    System.out.println("\nScenario2: Parallel ordering");
    List<String> data = Arrays.asList("A","B","C","D");
    System.out.print("forEach unordered: ");
    data.parallelStream().forEach(System.out::print);

    System.out.print("\nforEachOrdered: ");
    data.parallelStream().forEachOrdered(System.out::print);
    ;
    System.out.println();
}
```

Output

5.2 Optional in parallel stream

```
static void scenario3_optionalParallelStream() {
    System.out.println("\nScenario3: Optional + Parallel Stream");
    List<Optional<Integer>> numbers = Arrays.asList(
        Optional.of(1), Optional.empty(), Optional.of(4));
    boolean hasEven = numbers.parallelStream()
        .flatMap(Optional::stream)
        .anyMatch(n -> n % 2 == 0);
    System.out.println("Contains even? " + hasEven);
}
```

Output

5.3 Custom ForkJoinPool

```
static void scenario4_customForkJoinPool() {
    System.out.println("\nScenario4: Custom ForkJoinPool");
}
```

```

ForkJoinPool pool = new ForkJoinPool(4);
List<Integer> list = IntStream.rangeClosed(1, 1_000_000)
    .boxed().collect(Collectors.toList());
try {
    int sum = pool.submit(() -> list.parallelStream()
        .reduce(0, Integer::sum)).get();
    System.out.println("Sum with custom pool: " +
        sum);
} catch (Exception e) {
    e.printStackTrace();
}
}

```

Output

5.4 Parallel Map-Reduce large dataset

```

static void scenario6_parallelMapReduceLarge() {
    System.out.println("\nScenario6: Parallel map-reduce
        large dataset");
    List<Integer> nums = IntStream.rangeClosed(1, 10
        _000_000).boxed().collect(Collectors.toList());
    long start = System.nanoTime();
    long sum = nums.parallelStream().mapToLong(i -> i).sum
        ();
    long end = System.nanoTime();
    System.out.println("Sum: " + sum + ", Time(ms): " + (
        end-start)/1_000_000);
}

```

Output

5.5 Top-N elements using parallel stream

```

static void scenario8_topNParallel() {
    System.out.println("\nScenario8: Top-N parallel
        reduction");
    List<Integer> list = new Random().ints(1_000_000,1,1
        _000_000).boxed().collect(Collectors.toList());
    long start = System.nanoTime();
    List<Integer> top10 = list.parallelStream()
        .sorted(Comparator.reverseOrder())
        .limit(10)
        .collect(Collectors.toList());
    long end = System.nanoTime();
    System.out.println("Top 10 elements: " + top10 + ",
        Time(ms): " + (end-start)/1_000_000);
}

```

Output

5.6 Flatten nested Optional lists in parallel

```
static void scenario9_flatMapOptionalNested() {
    System.out.println("\nScenario9: Flatten nested
        Optional lists");
    List<Optional<List<String>>> nested = Arrays.asList(
        Optional.of(Arrays.asList("a","b")),
        Optional.empty(),
        Optional.of(Arrays.asList("c","d"))
    );
    List<String> flat = nested.parallelStream()
        .flatMap(Optional::stream)
        .flatMap(List::stream)
        .collect(Collectors.toList());
    System.out.println("Flattened list: " + flat);
}
```

Output

5.7 Parallel findFirst vs findAny

```
static void scenario10_parallelFindFirstVsFindAny() {
    System.out.println("\nScenario10: Parallel findFirst vs
        findAny");
    List<Integer> nums = IntStream.rangeClosed(1, 100).
        boxed().collect(Collectors.toList());
    Optional<Integer> first = nums.parallelStream().
        findFirst();
    Optional<Integer> any = nums.parallelStream().findAny();
    ;
    System.out.println("findFirst: " + first + ", findAny:
        " + any);
}
```

Output

5.8 Primitive vs boxed streams performance

```
static void scenario11_primitiveVsBoxedStream() {
    System.out.println("\nScenario11: Primitive vs Boxed
        Streams");
    long t1 = System.nanoTime();
    long sumBoxed = IntStream.rangeClosed(1,10_000_000).
        boxed().mapToLong(i->i).sum();
    long t2 = System.nanoTime();
    long sumPrimitive = IntStream.rangeClosed(1,10_000_000)
        .mapToLong(i->i).sum();
    long t3 = System.nanoTime();
    System.out.println("Boxed sum time(ms): " + (t2-t1)/1
        _000_000 + ", Primitive sum time(ms): " + (t3-t2)/1
        _000_000);
}
```

Output

5.9 Parallel filtering timing

```
static void scenario12_parallelFilteringTiming() {
    System.out.println("\nScenario12: Parallel filtering
        timing");
    List<Integer> list = IntStream.rangeClosed(1,5_000_000)
        .boxed().collect(Collectors.toList());
    long t1 = System.nanoTime();
    long countSeq = list.stream().filter(i->i%2==0).count()
        ;
    long t2 = System.nanoTime();
    long countPar = list.parallelStream().filter(i->i%2==0)
        .count();
    long t3 = System.nanoTime();
    System.out.println("Sequential filter count: " +
        countSeq + ", Time(ms): " + (t2-t1)/1_000_000);
    System.out.println("Parallel filter count: " + countPar
        + ", Time(ms): " + (t3-t2)/1_000_000);
}
```

Output

5.10 Parallel distinct performance

```
static void scenario13_parallelDistinctPerformance() {
    System.out.println("\nScenario13: Parallel distinct
        performance");
    List<Integer> list = IntStream.rangeClosed(1,1_000_000)
        .map(i->i%1000).boxed().collect(Collectors.toList());
    ;
    long t1 = System.nanoTime();
    long countSeq = list.stream().distinct().count();
    long t2 = System.nanoTime();
    long countPar = list.parallelStream().distinct().count
        ();
    long t3 = System.nanoTime();
    System.out.println("Sequential distinct count: " +
        countSeq + ", Time(ms): " + (t2-t1)/1_000_000);
    System.out.println("Parallel distinct count: " +
        countPar + ", Time(ms): " + (t3-t2)/1_000_000);
}
```

Output

5.11 Parallel sort timing

```
static void scenario14_parallelSortTiming() {
    System.out.println("\nScenario14: Parallel sort timing"
    );
    List<Integer> list = new Random().ints(5_000_000,1,10
    _000_000).boxed().collect(Collectors.toList());
    long t1 = System.nanoTime();
    list.stream().sorted().count();
    long t2 = System.nanoTime();
    list.parallelStream().sorted().count();
    long t3 = System.nanoTime();
    System.out.println("Sequential sort(ms): " + (t2-t1)/1
    _000_000 + ", Parallel sort(ms): " + (t3-t2)/1
    _000_000);
}
```

Output

5.12 Combined collectors with parallel stream

```
static void scenario15_combinedCollectorsAndParallel() {
    System.out.println("\nScenario15: Combined collectors +
    parallel stream");
    List<String> data = new Random().ints(1_000_000, 0,
    1000).mapToObj(String::valueOf).collect(Collectors.
    toList());
    Map<Integer, Long> result = data.parallelStream()
    .collect(Collectors.groupingByConcurrent(
    s -> Integer.parseInt(s)%10,
    Collectors.counting()
    ));
    System.out.println("Grouped count (mod 10): " + result)
    ;
}
```

Output

5.13 Nested Optional + parallel stream

```
static void scenario16_nestedOptionalParallel() {
    System.out.println("\nScenario16: Nested Optional +
    Parallel Stream");
    List<Optional<List<Optional<Integer>>>> nested = Arrays
    .asList(
    Optional.of(Arrays.asList(Optional.of(1), Optional.
    empty()))),
    Optional.empty(),
    Optional.of(Arrays.asList(Optional.of(2), Optional.of
    (3)))
    );
    List<Integer> flat = nested.parallelStream()
    .flatMap(Optional::stream)
    .flatMap(List::stream)
```

```
        .flatMap(Optional::stream)
        .collect(Collectors.toList());
        System.out.println("Flattened nested optionals: " +
            flat);
    }
```

Output

5.14 Parallel Top-N per group

```
static void scenario17_parallelTopNPerGroup() {
    System.out.println("\nScenario17: Parallel Top-N per
        group");
    Random r = new Random();
    List<Integer> nums = r.ints(1_000_000,1,100_000).boxed()
        .collect(Collectors.toList());
    Map<Integer, List<Integer>> top3PerMod = nums.
        parallelStream()
        .collect(Collectors.groupingByConcurrent(
            i -> i%10,
            Collectors.collectingAndThen(
                Collectors.toList(),
                list -> list.stream().sorted(Comparator.reverseOrder())
                    .limit(3).collect(Collectors.toList())
            )
        ));
    System.out.println("Top 3 per group (mod 10): " +
        top3PerMod);
}
```

Output

5.15 Custom ForkJoinPool with limited threads

```
static void scenario18_forkJoinCustomThreads() {
    System.out.println("\nScenario18: Custom ForkJoinPool
        limited threads");
    ForkJoinPool pool = new ForkJoinPool(2);
    List<Integer> nums = IntStream.rangeClosed(1, 1_000_000)
        .boxed().collect(Collectors.toList());
    try {
        long sum = pool.submit(() -> nums.
            parallelStream().mapToLong(i -> i).sum()).
            get();
        System.out.println("Sum with 2-thread custom
            pool: " + sum);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Output

5.16 Parallel anyMatch short-circuit

```
static void scenario19_parallelAnyMatchShortCircuit() {
    System.out.println("\nScenario19: Parallel anyMatch
short-circuit");
    List<Integer> nums = IntStream.rangeClosed(1, 10_000_000).boxed().collect(Collectors.toList());
    long start = System.nanoTime();
    boolean found = nums.parallelStream().anyMatch(i -> i
== 9_999_999);
    long end = System.nanoTime();
    System.out.println("Found 9_999_999? " + found + ",
Time(ms): " + (end-start)/1_000_000);
}
```

Output

5.17 Parallel allMatch timing

```
static void scenario20_parallelAllMatchTiming() {
    System.out.println("\nScenario20: Parallel allMatch
timing");
    List<Integer> nums = IntStream.rangeClosed(1, 5_000_000).boxed().collect(Collectors.toList());
    long startSeq = System.nanoTime();
    boolean allSeq = nums.stream().allMatch(i -> i < 6_000_000);
    long endSeq = System.nanoTime();
    long startPar = System.nanoTime();
    boolean allPar = nums.parallelStream().allMatch(i -> i
< 6_000_000);
    long endPar = System.nanoTime();
    System.out.println("Sequential allMatch: " + allSeq + "
, Time(ms): " + (endSeq-startSeq)/1_000_000);
    System.out.println("Parallel allMatch: " + allPar + "
Time(ms): " + (endPar-startPar)/1_000_000);
}
```

Output

5.18 Parallel partitioning collectors

```
static void scenario21_parallelPartitioningCollectors() {
    System.out.println("\nScenario21: Parallel partitioning
collectors");
    List<Integer> nums = IntStream.rangeClosed(1, 1_000_000).boxed().collect(Collectors.toList());
    Map<Boolean, Long> partitioned = nums.parallelStream().collect(Collectors.partitioningBy(i -> i%2==0,
Collectors.counting()));
    System.out.println("Partitioned even/odd counts: " +
partitioned);
}
```

Output

5.19 Parallel flatMap large nested lists

```
static void scenario22_parallelFlatMapLargeNestedLists() {
    System.out.println("\nScenario22: Parallel flatMap
        large nested lists");
    List<List<Integer>> nested = new ArrayList<>();
    for(int i=0;i<1000;i++){
        nested.add(IntStream.range(i*1000,(i+1)*1000).
            boxed().collect(Collectors.toList()));
    }
    long start = System.nanoTime();
    long count = nested.parallelStream().flatMap(List::
        stream).count();
    long end = System.nanoTime();
    System.out.println("Flattened element count: " + count
        + ", Time(ms): " + (end-start)/1_000_000);
}
```

Output

5.20 Parallel reduce with combiner

```
static void scenario23_parallelReduceWithCombiner() {
    System.out.println("\nScenario23: Parallel reduce with combiner
        ");
    List<String> strings = Arrays.asList("a","b","c","d","e","f","g
        ");
    String concat = strings.parallelStream().reduce("", (s1,s2)->s1
        +s2, (s1,s2)->s1+s2);
    System.out.println("Concatenated string: " + concat);
}
```

Output

5.21 Parallel stream peek for debugging

```
static void scenario24_parallelStreamPeekDebug() {
    System.out.println("\nScenario24: Parallel stream peek
        debugging");
    List<Integer> nums = IntStream.rangeClosed(1,10).boxed
        ().collect(Collectors.toList());
    nums.parallelStream()
        .peek(i -> System.out.println("Processing: " + i + "
            Thread: " + Thread.currentThread().getName()))
        .map(i -> i*2)
        .forEach(i -> {});
}
```

Output

5.22 Parallel stream with exception handling

```
static void scenario25_parallelStreamWithExceptionHandling() {
    System.out.println("\nScenario25: Parallel stream with
        exception handling");
    List<Integer> nums = Arrays.asList(1,2,0,4);
    nums.parallelStream().forEach(i -> {
        try {
            int res = 10 / i;
            System.out.println("10 / " + i + " = "
                + res);
        } catch (ArithmeticException e) {
            System.out.println("Caught division by
                zero for i=" + i);
        }
    });
}
```

Output

Chapter 6

Real time Streams Demo

6.1 Model class used for coming questions

```
static class User {
    String name;
    boolean active;
    Date lastLogin;
    String email;
    String city;
    User(String n, boolean a, String d, String e, String c)
        throws Exception {
        name=n; active=a; lastLogin=new
            SimpleDateFormat("yyyy-MM-dd").parse(d);
            email=e; city=c;
    }
    public String toString(){return name+"("+city+")";}
}

static class Transaction {
    String user; double amount;
    Transaction(String u, double a){user=u; amount=a;}
}

static class Product {
    String name; int stock; int sold;
    Product(String n, int s, int so){name=n; stock=s; sold=
        so;}
}

static class Employee {
    String name; String dept; double salary;
    Employee(String n,String d,double s){name=n;dept=d;
        salary=s;}
    public String toString(){return name+"-"+salary;}
}

static class Order {
    String customer; List<String> items;
    Order(String c, List<String> i){customer=c; items=i;}
}

static class Student {
    String name; List<Integer> scores;
```

```
        Student(String n,List<Integer> s){name=n; scores=s;}  
    }
```

6.2 Active users sorted by last login

```
List<User> users = Arrays.asList(  
    new User("Alice", true, "2026-02-25", "alice@xyz.com", "Mumbai"),  
    new User("Bob", false, "2026-01-12", "bob@xyz.com", "Delhi"),  
    new User("Charlie", true, "2026-02-28", "charlie@xyz.com", "Mumbai"  
    )  
);  
System.out.println("1. Active users sorted by last login:");  
users.stream()  
    .filter(u->u.active)  
    .sorted((u1,u2)->u2.lastLogin.compareTo(u1.lastLogin))  
    .forEach(System.out::println);
```

Output

6.3 Total transaction amount per user

```
List<Transaction> txns = Arrays.asList(  
    new Transaction("Alice",125.50),  
    new Transaction("Bob",250.00),  
    new Transaction("Alice",75.25)  
);  
System.out.println("\n2. Total spend per user:");  
Map<String, Double> totalPerUser =  
    txns.stream().collect(Collectors.groupingBy(  
        t->t.user,  
        Collectors.summingDouble(t->t.amount)  
    ));  
System.out.println(totalPerUser);
```

Output

6.4 Out of stock products

```
List<Product> products = Arrays.asList(  
    new Product("Laptop",10, 50),  
    new Product("Mouse",0, 100),  
    new Product("Keyboard",5,30)  
);  
System.out.println("\n3. Out of stock products:");  
products.stream().filter(p->p.stock==0).forEach(p->System.out.  
    println(p.name));
```

Output

6.5 Highest salary by department

```
List<Employee> emps = Arrays.asList(  
    new Employee("Alice", "HR", 50000),  
    new Employee("Bob", "IT", 75000),  
    new Employee("Charlie", "IT", 80000)  
);  
System.out.println("\n4. Highest salary in each dept:");  
Map<String, Optional<Employee>> highestSal =  
    emps.stream().collect(Collectors.groupingBy(  
        e->e.dept,  
        Collectors.maxBy(Comparator.comparingDouble(x->x.salary))  
    ));  
System.out.println(highestSal);
```

Output

6.6 Customers with >1 orders

```
List<Order> orders = Arrays.asList(  
    new Order("Alice", Arrays.asList("Laptop")),  
    new Order("Bob", Arrays.asList("Mouse", "Keyboard")),  
    new Order("Alice", Arrays.asList("Monitor"))  
);  
System.out.println("\n5. Customers with more than 1 order:");  
orders.stream()  
    .collect(Collectors.groupingBy(o->o.customer, Collectors.  
        counting()))  
    .entrySet().stream()  
    .filter(e->e.getValue()>1)  
    .forEach(System.out::println);
```

Output

6.7 Comma-separated emails

```
System.out.println("\n6. All user emails:");  
String emails = users.stream().map(u->u.email).collect(  
    Collectors.joining(", "));  
System.out.println(emails);
```

Output

6.8 Average student scores

```
List<Student> students = Arrays.asList(  
    new Student("Alice", Arrays.asList(85, 90, 78)),  
    new Student("Bob", Arrays.asList(80, 70, 88))  
);  
System.out.println("\n7. Average scores:");
```

```
students.stream().forEach(s-> {  
    double avg = s.scores.stream().mapToInt(i->i).average().orElse  
        (0);  
    System.out.println(s.name+" -> "+avg);  
});
```

Output

6.9 Distinct cities

```
System.out.println("\n8. Distinct cities:");  
users.stream().map(u->u.city).distinct().forEach(System.out::println);
```

Output

6.10 Flatten all purchased items

```
System.out.println("\n9. All purchased items:");  
orders.stream()  
    .flatMap(o->o.items.stream())  
    .distinct()  
    .forEach(System.out::println);
```

Output

6.11 Top 3 best selling products

```
System.out.println("\n10. Top 3 best selling products:");  
products.stream()  
    .sorted((p1,p2)->Integer.compare(p2.sold, p1.sold))  
    .limit(3)  
    .forEach(p->System.out.println(p.name + " sold="+p.sold));
```

Output

6.12 Initialize Dataset

```
// --- Domain Classes ---  
static class Customer {  
    String name;  
    int age;  
    String city;  
  
    Customer(String n, int a, String c) {  
        name = n;  
        age = a;  
        city = c;  
    }  
}
```



```
    }

    public String toString() {
        return name + "(" + city + ")";
    }
}

static class Account {
    String accNo;
    String custName;
    double balance;
    String type;

    Account(String a, String c, double b, String t) {
        accNo = a;
        custName = c;
        balance = b;
        type = t;
    }

    public String toString() {
        return accNo + "-" + custName + "-" + balance +
            "-" + type;
    }
}

static class Transaction {
    String id;
    String accNo;
    double amount;
    String type;

    Transaction(String i, String a, double am, String t) {
        id = i;
        accNo = a;
        amount = am;
        type = t;
    }

    public String toString() {
        return id + "-" + accNo + "-" + type + "-" +
            amount;
    }
}

static class Product {
    String name;
    int stock;
    int sold;
    double price;

    Product(String n, int s, int so, double p) {
        name = n;
        stock = s;
        sold = so;
        price = p;
    }
}
```

```

        public String toString() {
            return name + "-" + stock + "-" + sold + "-" +
                price;
        }
    }

    static class Employee {
        String name;
        String dept;
        double salary;

        Employee(String n, String d, double s) {
            name = n;
            dept = d;
            salary = s;
        }

        public String toString() {
            return name + "-" + dept + "-" + salary;
        }
    }

    static class Student {
        String name;
        List<Integer> grades;

        Student(String n, List<Integer> g) {
            name = n;
            grades = g;
        }

        public String toString() {
            return name + "-" + grades;
        }
    }
}

```

Output

```

// =====
// Initialize Realistic Datasets
// =====

// Banking Customers
List<Customer> customers = Arrays.asList(
    new Customer("Alice", 30, "Mumbai"),
    new Customer("Bob", 45, "Delhi"),
    new Customer("Charlie", 28, "Mumbai"),
    new Customer("David", 50, "Bangalore"),
    new Customer("Eve", 35, "Delhi"),
    new Customer("Frank", 55, "Mumbai")
);

// Accounts
List<Account> accounts = Arrays.asList(
    new Account("A101", "Alice", 5000, "SAVINGS"),
    new Account("A102", "Bob", 15000, "CURRENT"),
    new Account("A103", "Charlie", 2000, "SAVINGS"),
    new Account("A104", "David", 25000, "CURRENT"),

```

```

new Account("A105", "Eve", 12000, "SAVINGS"),
new Account("A106", "Frank", 3000, "SAVINGS")
);

// Transactions
List<Transaction> transactions = Arrays.asList(
new Transaction("T1", "A101", 2000, "DEPOSIT"),
new Transaction("T2", "A102", 5000, "WITHDRAWAL"),
new Transaction("T3", "A101", 1000, "WITHDRAWAL"),
new Transaction("T4", "A103", 1500, "DEPOSIT"),
new Transaction("T5", "A104", 7000, "DEPOSIT"),
new Transaction("T6", "A105", 3000, "WITHDRAWAL"),
new Transaction("T7", "A106", 4000, "DEPOSIT"),
new Transaction("T8", "A102", 2000, "DEPOSIT")
);

// E-commerce Products
List<Product> products = Arrays.asList(
new Product("Laptop", 10, 50, 50000),
new Product("Mouse", 0, 100, 500),
new Product("Keyboard", 5, 30, 1500),
new Product("Monitor", 2, 20, 12000),
new Product("Headset", 15, 40, 2500)
);

// HR Employees
List<Employee> employees = Arrays.asList(
new Employee("Alice", "HR", 50000),
new Employee("Bob", "IT", 75000),
new Employee("Charlie", "IT", 80000),
new Employee("David", "Finance", 60000),
new Employee("Eve", "HR", 55000),
new Employee("Frank", "Finance", 65000)
);

// Education Students
List<Student> students = Arrays.asList(
new Student("Alice", Arrays.asList(85, 90, 78)),
new Student("Bob", Arrays.asList(80, 70, 88)),
new Student("Charlie", Arrays.asList(90, 95, 92)),
new Student("David", Arrays.asList(60, 75, 70)),
new Student("Eve", Arrays.asList(88, 92, 85))
);

```

Output

6.13 Distinct cities

```

System.out.println("\n8. Distinct cities:");
users.stream().map(u->u.city).distinct().forEach(System.out::
println);

```

Output

6.14 Banking Examples

```
//Find the oldest customer (max age using streams)
System.out.println("\n1. Oldest customer:");
customers.stream()
    .max(Comparator.comparingInt(c -> c.age))
    .ifPresent(System.out::println);

//Accounts with balance > 10,000 (filter)
System.out.println("\n2. Accounts with balance>10000:");
accounts.stream()
    .filter(a -> a.balance > 10000)
    .forEach(System.out::println);

//Total bank balance (sum using mapToDouble)
System.out.println("\n3. Total bank balance:");
double totalBalance = accounts.stream()
    .mapToDouble(a -> a.balance)
    .sum();
System.out.println(totalBalance);

//Max transaction amount (find max)
System.out.println("\n4. Max transaction amount:");
transactions.stream()
    .max(Comparator.comparingDouble(t -> t.amount))
    .ifPresent(System.out::println);

//Transactions grouped by type (groupingBy)
System.out.println("\n5. Transactions grouped by type:");
Map<String, List<Transaction>> txByType = transactions.stream()
    .collect(Collectors.groupingBy(t -> t.type));
System.out.println(txByType);

//Customers with multiple accounts (counting & filtering)
System.out.println("\n6. Customers with multiple accounts:");
Map<String, Long> accCount = accounts.stream()
    .collect(Collectors.groupingBy(a -> a.custName, Collectors.counting()));
accCount.entrySet().stream().filter(e -> e.getValue() > 1).forEach(
    System.out::println);

//Average account balance
System.out.println("\n7. Average account balance:");
double avgBalance = accounts.stream().mapToDouble(a -> a.balance)
    .average().orElse(0);
System.out.println(avgBalance);

//Accounts sorted descending by balance
System.out.println("\n8. Accounts sorted descending by balance:");
accounts.stream()
    .sorted((a1, a2) -> Double.compare(a2.balance, a1.balance))
    .forEach(System.out::println);

//Total deposits (filter + sum)
System.out.println("\n9. Total deposits:");
double totalDeposits = transactions.stream()
    .filter(t -> t.type.equals("DEPOSIT"))
    .mapToDouble(t -> t.amount)
```

```

.sum();
System.out.println(totalDeposits);

//Customers with balance > 12,000 (map + collect)
System.out.println("\n10. Customers with balance>12000:");
Set<String> richCustomers = accounts.stream()
    .filter(a -> a.balance > 12000)
    .map(a -> a.custName)
    .collect(Collectors.toSet());
System.out.println(richCustomers);

```

Output

6.15 E-commerce Examples

```

//Out-of-stock products
System.out.println("\n11. Out-of-stock products:");
products.stream().filter(p -> p.stock == 0).forEach(System.out
    ::println);

//Top selling product
System.out.println("\n12. Top selling product:");
products.stream().max(Comparator.comparingInt(p -> p.sold)).
    ifPresent(System.out::println);

//Total revenue per product (sold*price)
System.out.println("\n13. Total revenue per product:");
products.forEach(p -> System.out.println(p.name + " -> " + (p.
    sold * p.price)));

//Products with low stock <5
System.out.println("\n14. Products with stock<5:");
products.stream().filter(p -> p.stock < 5).forEach(System.out::
    println);

//Average price
System.out.println("\n15. Average price of products:");
System.out.println(products.stream().mapToDouble(p -> p.price).
    average().orElse(0));

//Product names joined
System.out.println("\n16. Product names joined:");
System.out.println(products.stream().map(p -> p.name).collect(
    Collectors.joining(", ")));

//Products sorted by revenue descending
System.out.println("\n17. Products sorted by revenue descending
    :");
products.stream().sorted((p1, p2) -> Double.compare(p2.sold *
    p2.price, p1.sold * p1.price))
    .forEach(System.out::println);

//Total stock remaining
System.out.println("\n18. Total stock remaining:");
System.out.println(products.stream().mapToInt(p -> p.stock).sum
    ());

```

```
//Total sold units
System.out.println("\n19. Total sold units:");
System.out.println(products.stream().mapToInt(p -> p.sold).sum
());

// Products with revenue>50,000
System.out.println("\n20. Products with revenue>50000:");
products.stream().filter(p -> p.sold * p.price > 50000).forEach
(System.out::println);
```

Output

Id	Make/Model	# Wheels	# Doors	Type
1	Ford Focus	4	4	Sedan
2	Tesla Roadster	4	2	Sports
3	Kawakasi Ninja	2	0	Motorcycle
4	McLaren Formula 1	4	0	Race
5	Tesla S	4	4	Sedan

Table 6.1: Vehicle Table Example